

Generative AI using Large Language Models (LLMs)

Lecture 2 | 01/31/25

Instructor: Chen Wang
Fri 4:10 pm - 6:40 pm | 517 Hamilton Hall

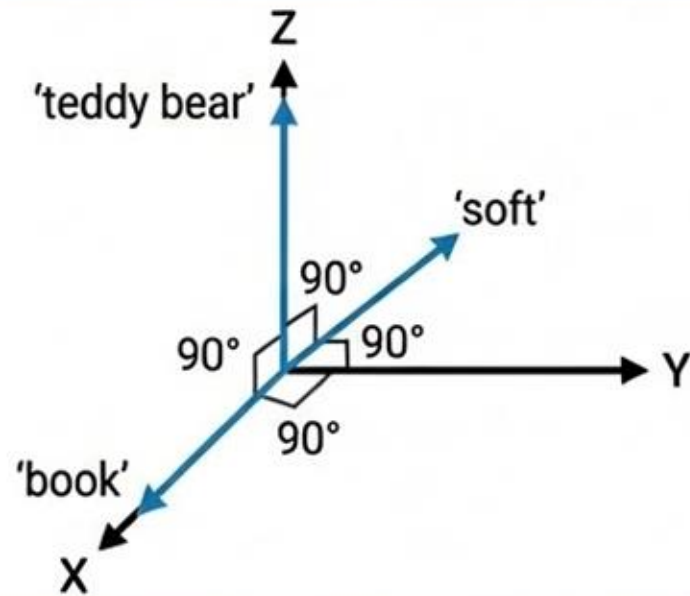
Agenda

- Word embeddings
- Attention mechanism
- Transformer architecture
- Mixture of Experts (MoE)
- Encoder only vs Decoder only Models
 - BERT, GPT, LLAMA



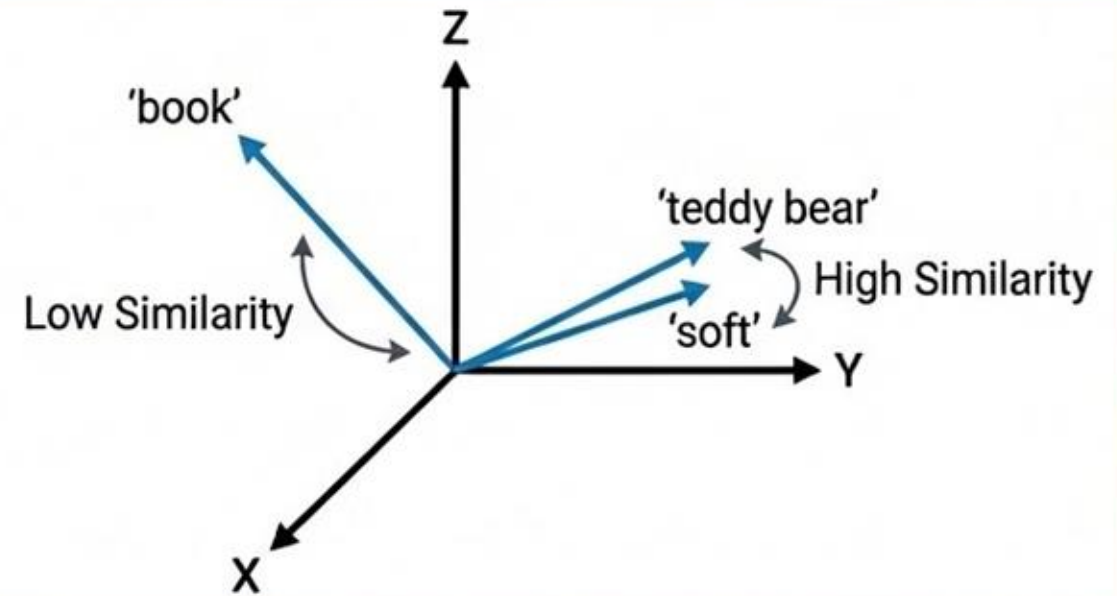
From Words to Numbers: The Evolution of Embeddings

1-Hot Representation: Orthogonal & Naive



Each word is an isolated vector.
All vectors are at 90° angles (orthogonal).
'teddy bear' is equally unrelated to 'soft' and 'book'.

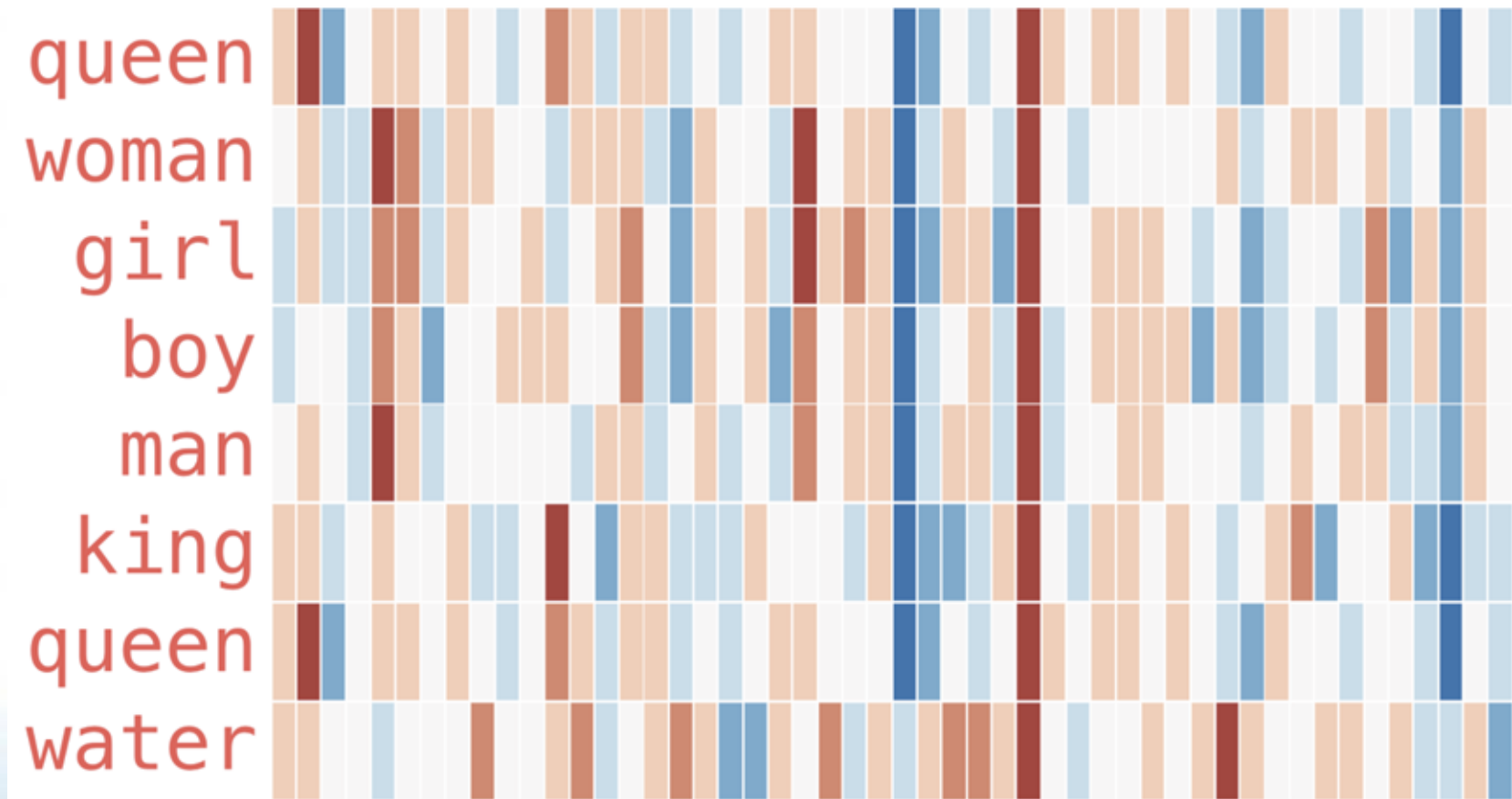
Word Embedding: Semantic Similarity



Words are mapped to a vector space where distance reflects meaning.
'teddy bear' is physically closer to 'soft' than 'book', capturing their relationship.



Word2Vec



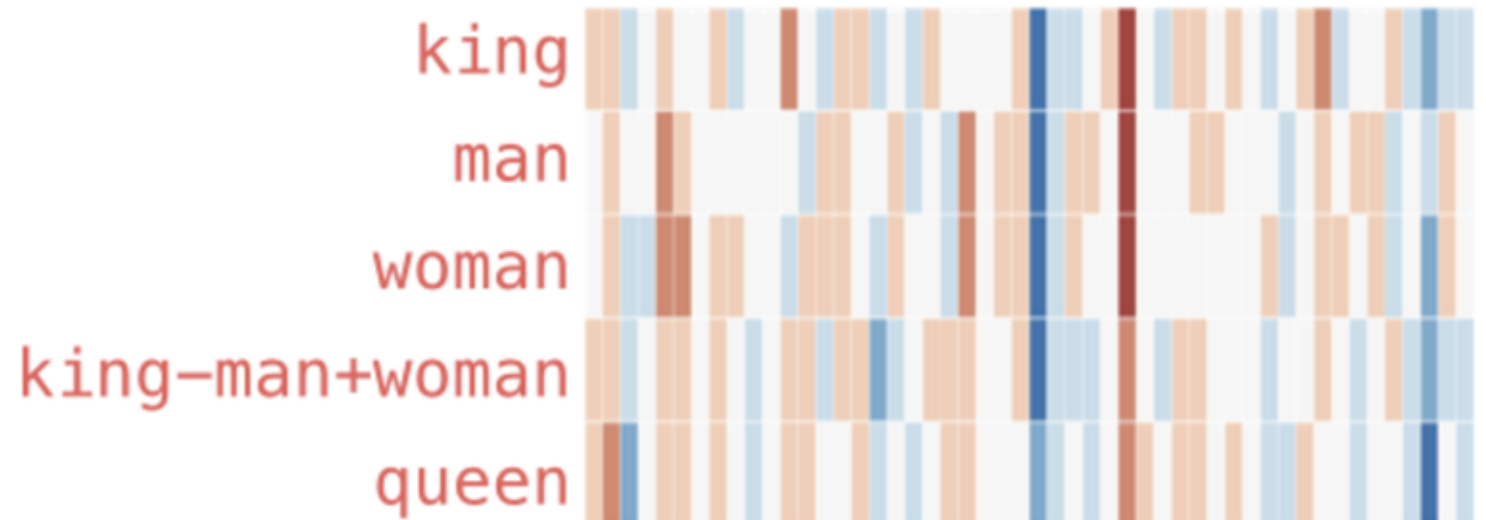


Operations using word embeddings

```
model.most_similar(positive=["king", "woman"], negative=["man"])
```

```
[('queen', 0.8523603677749634),  
 ('throne', 0.7664333581924438),  
 ('prince', 0.7592144012451172),  
 ('daughter', 0.7473883032798767),  
 ('elizabeth', 0.7460219860076904),  
 ('princess', 0.7424570322036743),  
 ('kingdom', 0.7337411642074585),  
 ('monarch', 0.721449077129364),  
 ('eldest', 0.7184862494468689),  
 ('widow', 0.7099430561065674)]
```

king - man + woman $\approx$ queen





Problem with static embeddings (word2vec)

- A major limitation is that embeddings are static; the embedding for a word doesn't reflect how its meaning changes in context, like a chameleon.

The chicken didn't cross the road because it was too tired.

In Word2Vec, the word "it" has one fixed vector, but here it is ambiguous. Does "it" refer to the chicken or the road? A static embedding can't distinguish between the two; it just knows "it" is a pronoun.



Contextual Embeddings

Intuition: a representation of meaning of a word should be different in different contexts!

Contextual Embedding: each word has a different vector that expresses different meanings depending on the surrounding words



Contextual Embeddings

The chicken didn't cross the road because it

What should be the properties of "it"?

The chicken didn't cross the road because it was too **tired**

The chicken didn't cross the road because it was too **wide**

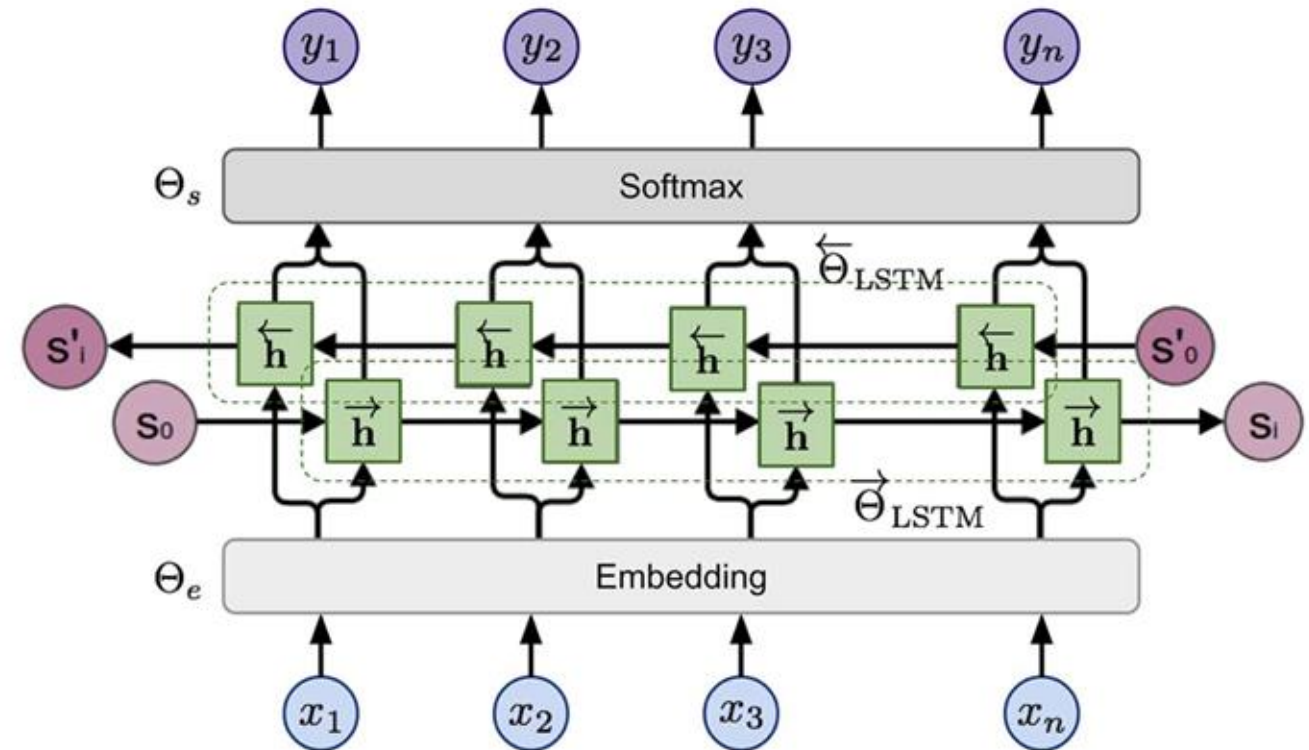
At this point in the sentence, it's probably referring to either the chicken or the street

But the static meaning of "it" can't represent that, it just means "I am a non-human pronoun"



ELMo: Context Matters

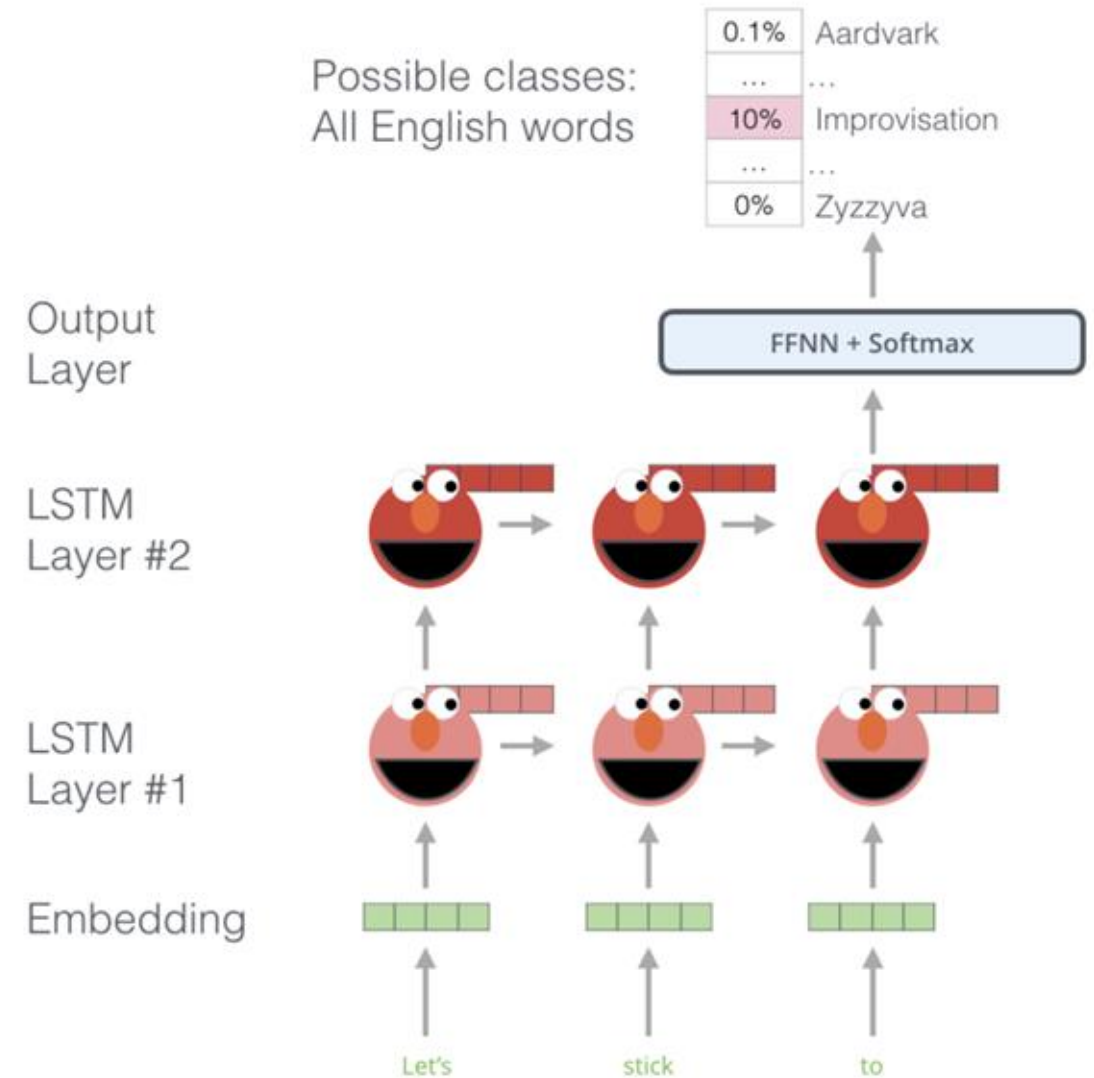
- A word can have different meanings depending on the context
 - Give me the stick
 - Let's stick to improving our work
- ELMo generates *contextualized* embeddings for a word
- Contextualized word-embeddings can give words different embeddings based on the meaning they carry in the context of the sentence
- LSTM based architecture





ELMo Training

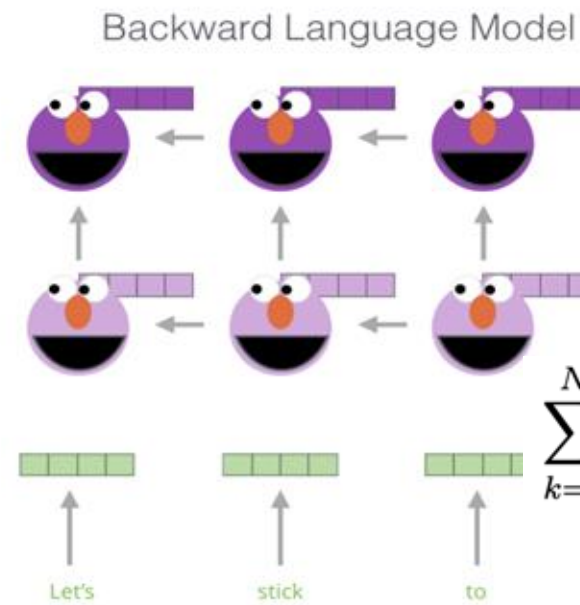
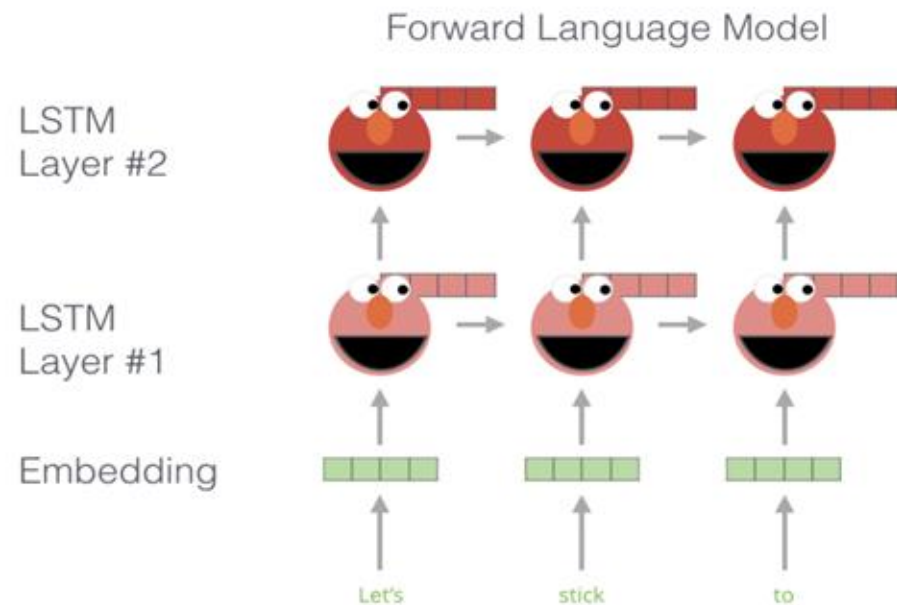
- Predict next word in a sequence of words
- Self-supervised learning, without any need for labels





Embedding generation in ELMo: Step 1

Embedding of “stick” in “Let’s stick to” - Step #1



Training objective: Jointly maximizes the log likelihood of the forward and backward directions

$$\sum_{k=1}^N \left(\log p(t_k | t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{LSTM}, \Theta_s) + \log p(t_k | t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s) \right)$$

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_1, t_2, \dots, t_{k-1})$$

$$p(t_1, t_2, \dots, t_N) = \prod_{k=1}^N p(t_k | t_{k+1}, t_{k+2}, \dots, t_N)$$

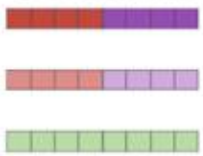


Embedding generation in ELMo: Step 2

- Contextualized embedding through grouping together the hidden states (and initial embedding)

Embedding of “stick” in “Let’s stick to” - Step #2

1- Concatenate hidden layers



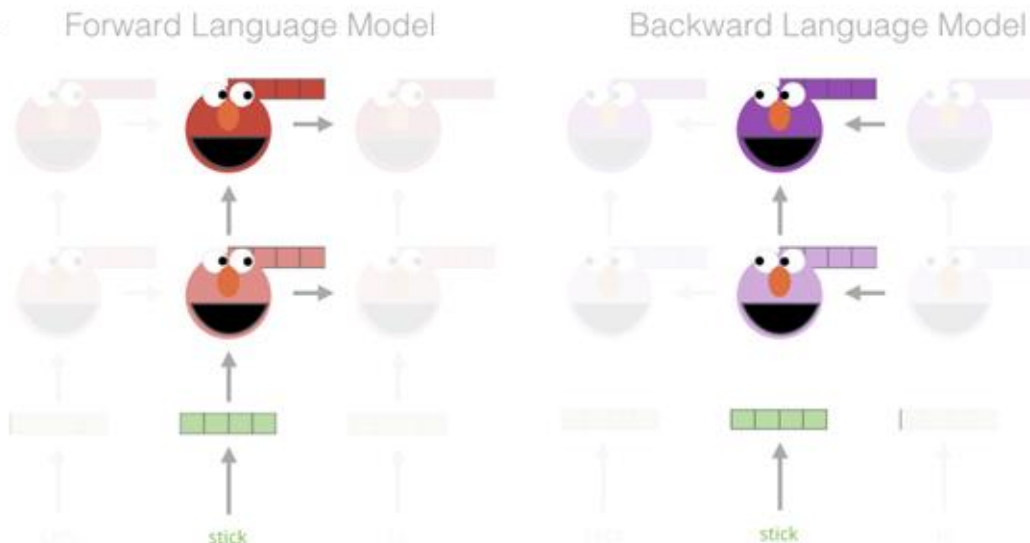
2- Multiply each vector by a weight based on the task



3- Sum the (now weighted) vectors



ELMo embedding of “stick” for this task in this context



each token t_k , a L -layer biLM computes a set of $2L + 1$ representations

$$\begin{aligned} R_k &= \{\mathbf{x}_k^{LM}, \vec{\mathbf{h}}_{k,j}^{LM}, \overleftarrow{\mathbf{h}}_{k,j}^{LM} \mid j = 1, \dots, L\} \\ &= \{\mathbf{h}_{k,j}^{LM} \mid j = 0, \dots, L\}, \end{aligned}$$

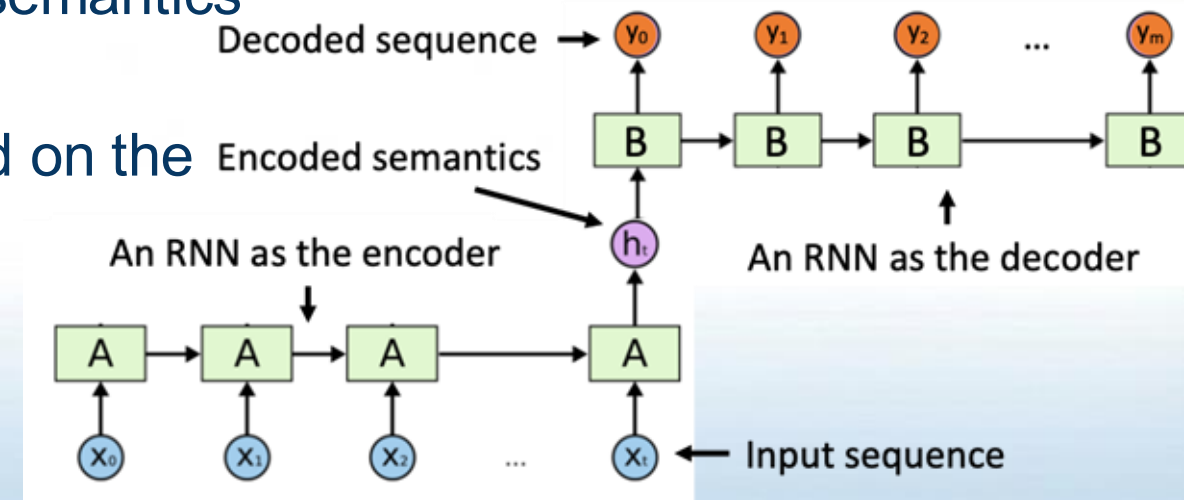
Task specific ELMo embedding

$$\mathbf{ELMo}_k^{task} = E(R_k; \Theta^{task}) = \gamma^{task} \sum_{j=0}^L s_j^{task} \mathbf{h}_{k,j}^{LM}$$



Sequence-to-Sequence (Seq2Seq) model

- Developed by Google in 2018 for use in machine translation.
- Seq2seq turns one sequence into another sequence. It does so by use of a recurrent neural network (RNN) or more often LSTM or GRU to avoid the problem of vanishing gradient.
- The primary components are one Encoder and one Decoder network. The encoder turns each item into a corresponding hidden vector containing the item and its context. The decoder reverses the process, turning the vector into an output item, using the previous output as the input context.
- **Encoder RNN:** extract and compress the semantics from the input sequence
- **Decoder RNN:** generate a sequence based on the input semantics
- Apply to tasks such as machine translation
 - Similar underlying semantics
 - E.g., “I love you.” to “Je t’aime.”

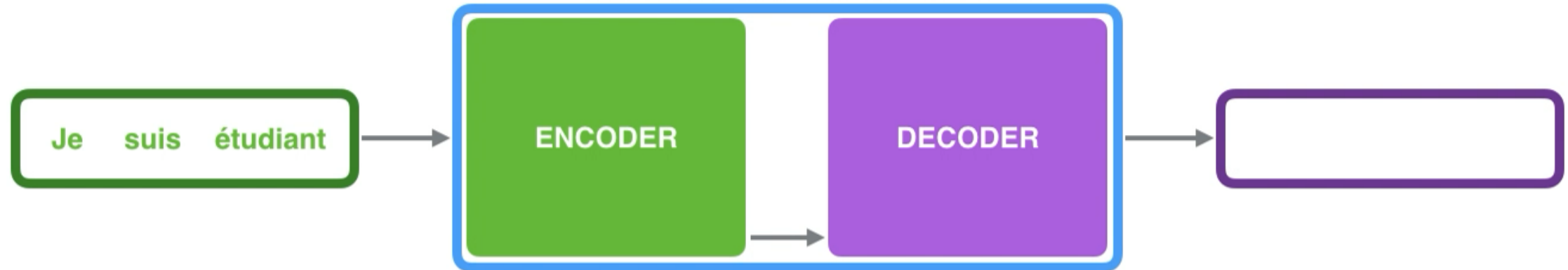




Context in Seq2Seq model

Time step:

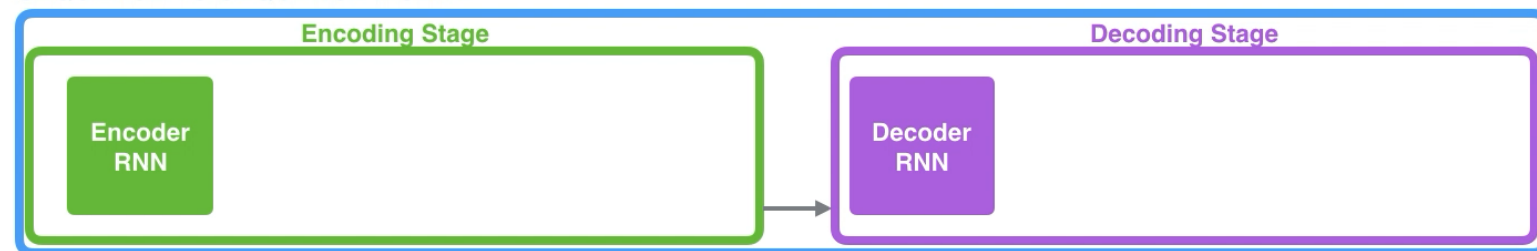
Neural Machine Translation
SEQUENCE TO SEQUENCE MODEL





Hidden states as context

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



The last **hidden state** of the **encoder** forms the **context** we pass along to the **decoder**.

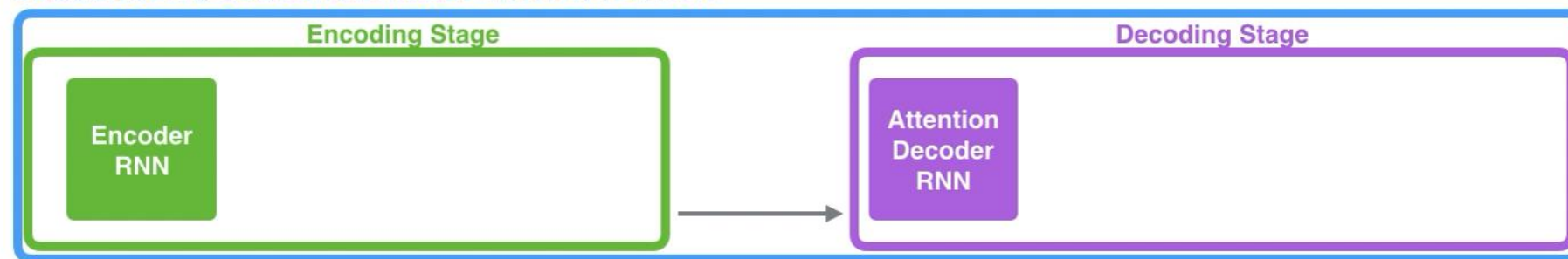
Je suis étudiant



Hidden states as context

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



The **decoder** also maintains a **hidden state** that it passes from one time step to the next.

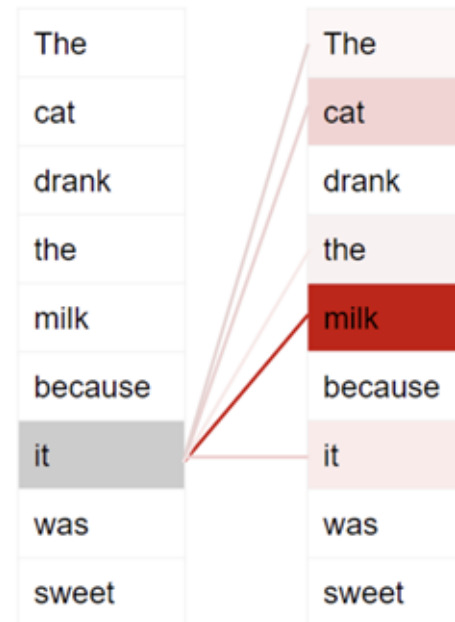
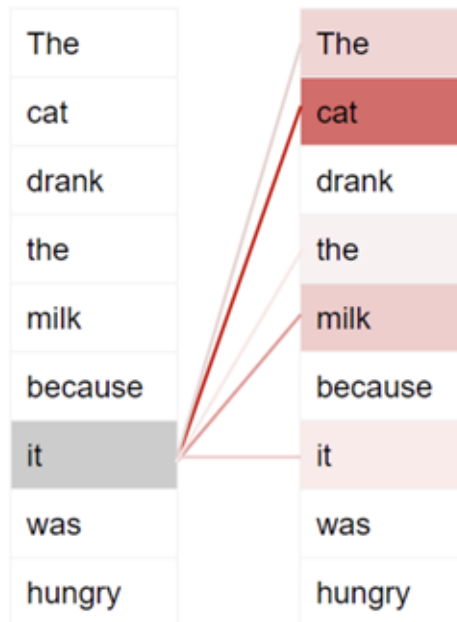
Je suis étudiant

Attention



Motivating Attention

- Capture long-range dependence in context
- Example:
 1. The *cat* drank the milk because *it* was hungry.
 2. The cat drank the *milk* because *it* was sweet.





Attention

- The **context** vector turned out to be a bottleneck for Seq2Seq models
- “Attention” mechanism was first proposed in [Bahdanau et al., 2014](#) and [Luong et al., 2015](#)
- Attention allows a Seq2Seq model to focus on the relevant parts of the input sequence when decoding
- Attention model differs from a classic Seq2Seq model:
 1. The **encoder** passes *all* the **hidden states** to the **decoder**
 2. The **decoder** gives different scores to different hidden states from the **encoder**



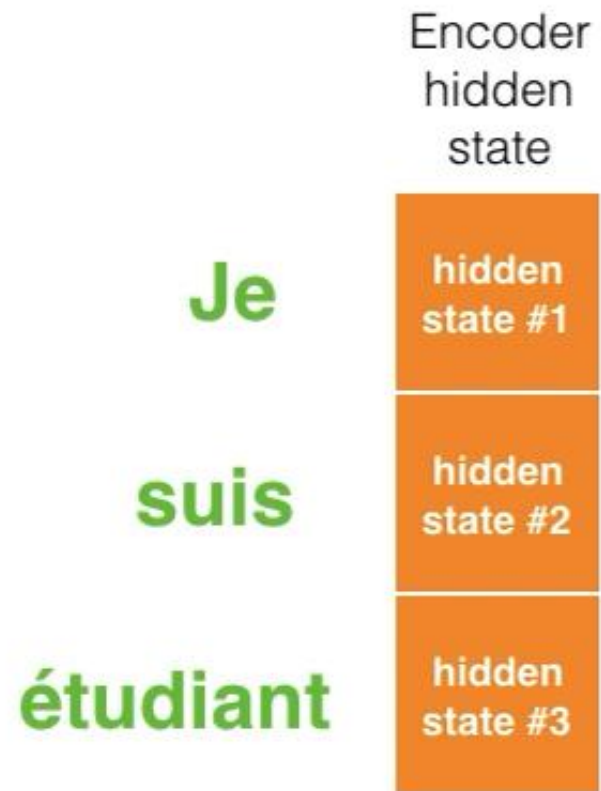
Attention Mechanism Steps

Attention at time step 4



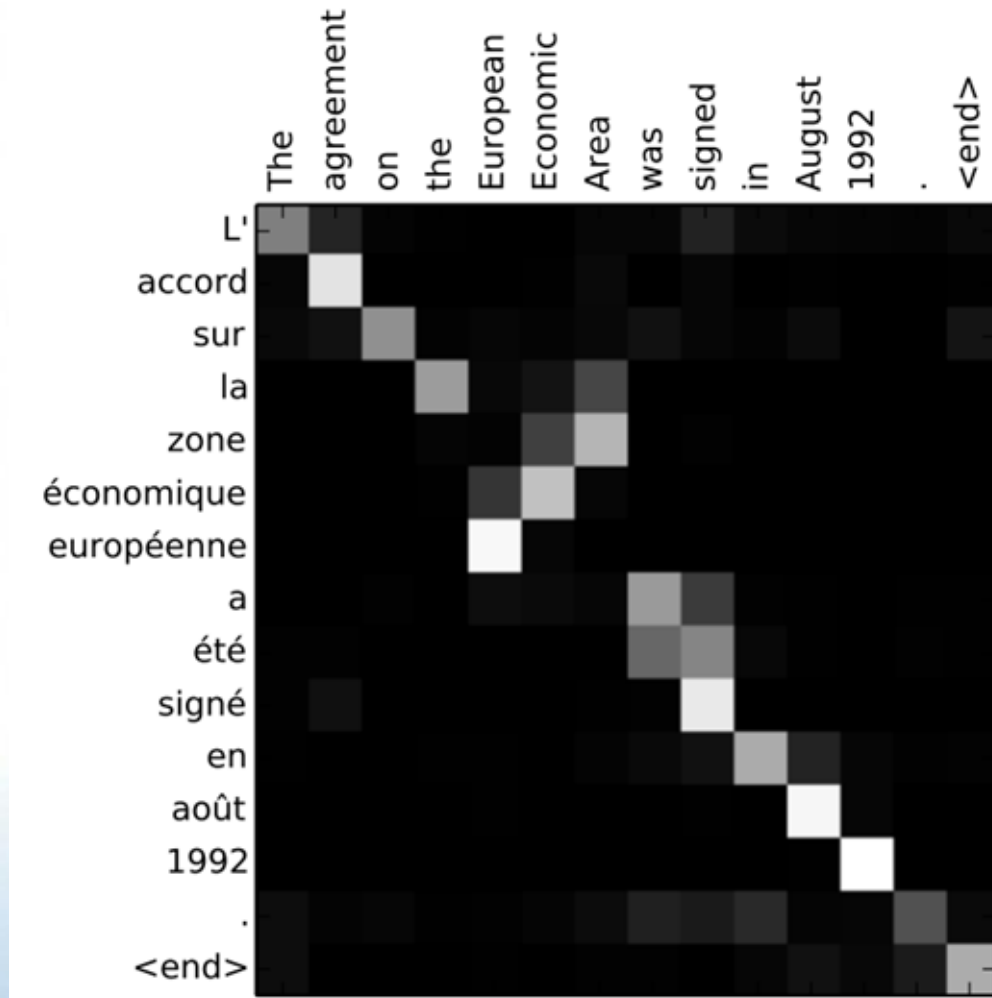


Attention example





Attention example



- **Attention Weights:** Brightness indicates the model's focus (white = high focus, black = low focus).
- **Linear Translation:** A crisp diagonal line shows word-for-word translation in the same order.
- **Handling Word Order:** The model breaks the diagonal pattern for language differences, such as noun-adjective order.
- **Dynamic Reordering:** The model "jumps forward" to translate the noun first, then "looks back" to retrieve adjectives.
- **Learning Grammar:** This proves the model learns grammatical rules and reorders information dynamically, not just blindly replacing words.

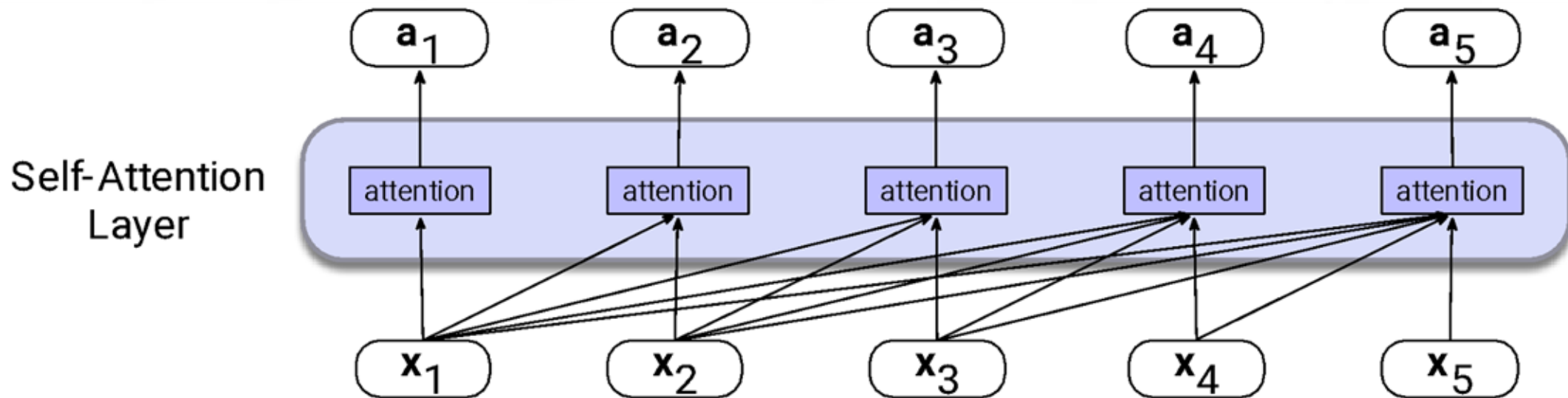


Attention Definition

- A mechanism
 - for helping compute **the embedding for a token** by selectively attending to and integrating information from surrounding tokens (at the previous layer).
- More formally:
 - a method for doing a **weighted sum of vectors**.



Left-to-right Attention





Simplified Version of Attention

A sum of prior words weighted by their similarity with the current word

Given a sequence of token embeddings:

$x_1 \quad x_2 \quad x_3 \quad x_4 \quad x_5 \quad x_6 \quad x_7 \quad x_i$

Produce: a_i = a weighted sum of x_1 through x_7 (and x_i)

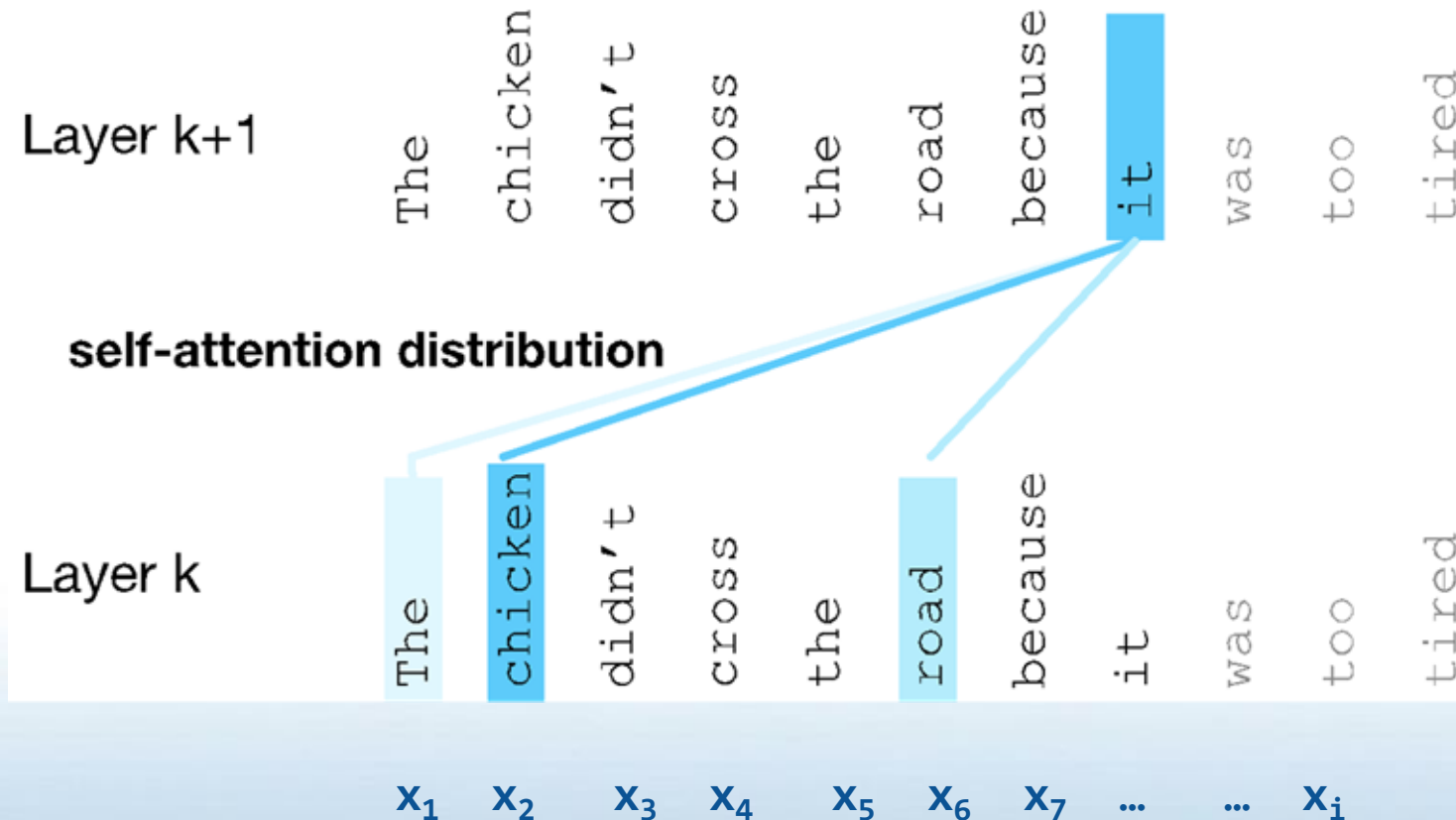
Weighted by their similarity to x_i

$$\begin{aligned} \text{score}(x_i, y_j) &= x_i \dot{y}_j \\ a_{ij} &= \text{softmax}(\text{score}(x_j, y_j)), \quad j \leq i \\ \mathbf{a}_i &= \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \end{aligned}$$



Intuition of attention

columns corresponding to input tokens





An Actual Attention Head

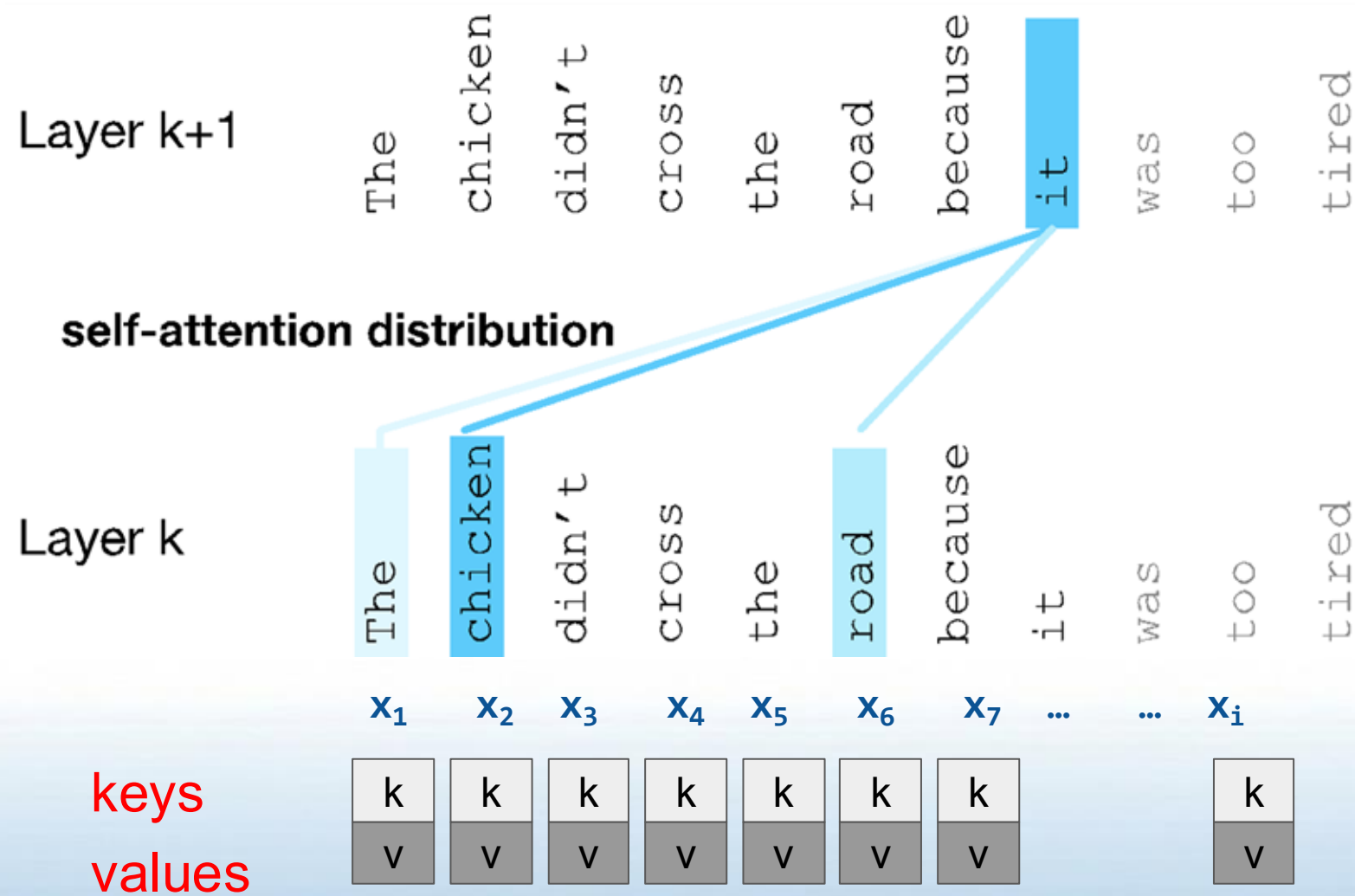
slightly more complicated

High-level idea: instead of using vectors (like x_i and x_4) directly, we'll represent 3 separate roles each vector x_i plays:

- **query:** As the current element being compared to the preceding inputs.
- **key:** as a preceding input that is being compared to the current element to determine a similarity
- **value:** a value of a preceding element that gets weighted and summed



Attention Intuition





An Actual Attention Head

slightly more complicated

We'll use matrices to project each vector x_i into a representation of its role as query, key, value:

- query: W^Q
- key: W^K
- value: W^V

$$\mathbf{q}_i = \mathbf{x}_i W^Q; \quad \mathbf{k}_i = \mathbf{x}_i W^K; \quad \mathbf{v}_i = \mathbf{x}_i W^V$$



An Actual Attention Head

slightly more complicated

- Given these 3 representation of x_i

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i \mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i \mathbf{W}^V$$

- To compute similarity of current element x_i with some prior element x_j
- We'll use dot product between $\mathbf{q}_i$ and $\mathbf{k}_j$.
- And instead of summing up x_j , we'll sum up $\mathbf{v}_j$



Final equations for one attention head

$$\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q; \quad \mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K; \quad \mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$$

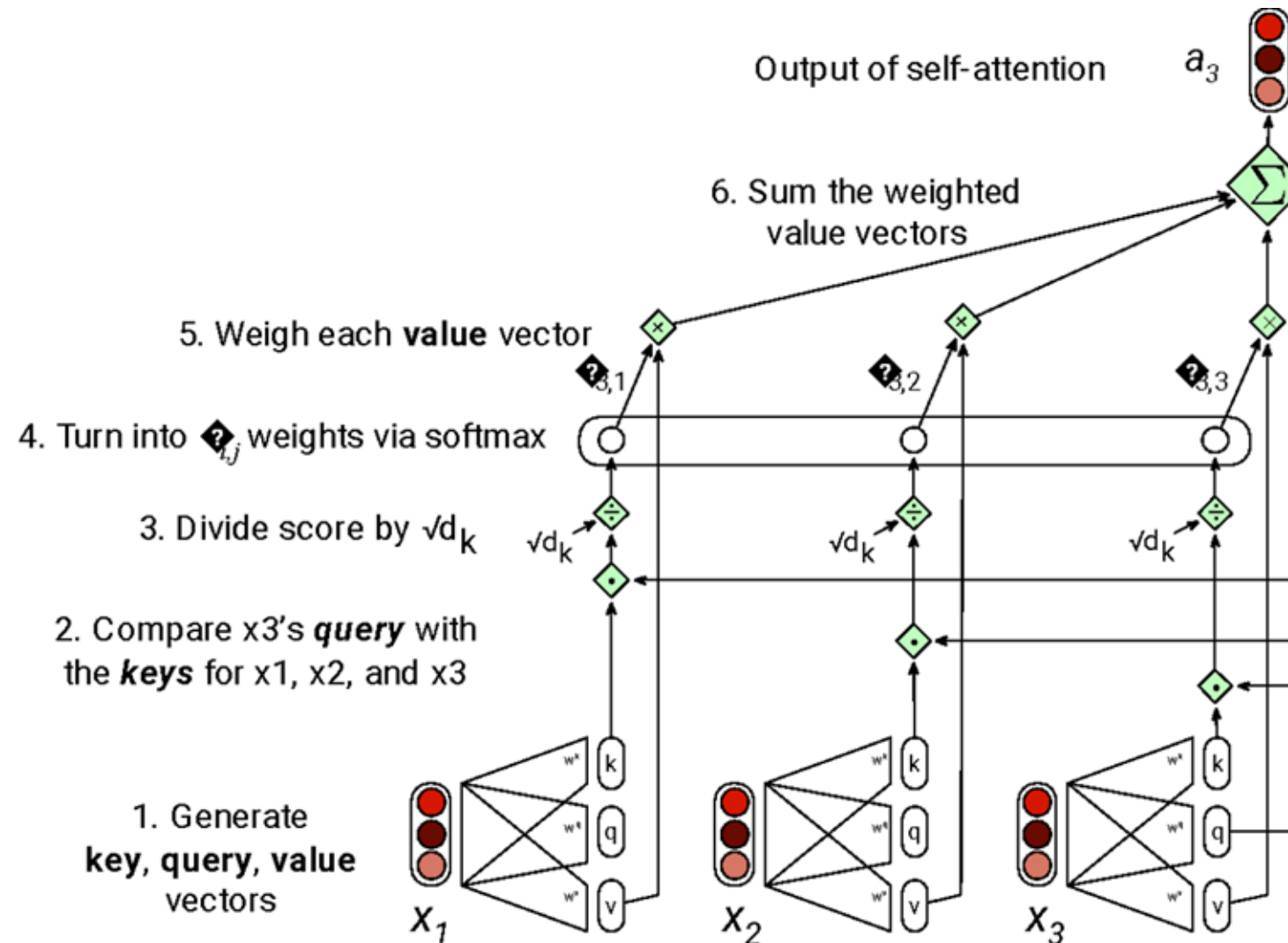
$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$



Calculating the value of a_3





Actual Attention

slightly more complicated

- Instead of one attention head, we'll have lots of them!
- Intuition: each head might be attending to the context for different purposes
 - Different linguistic relationships or patterns in the context

$$\mathbf{q}_i^c = \mathbf{x}_i \mathbf{W}^{\mathbf{Q}^c}; \quad \mathbf{k}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{K}^c}; \quad \mathbf{v}_j^c = \mathbf{x}_j \mathbf{W}^{\mathbf{V}^c}; \quad \forall c \quad 1 \leq c \leq h$$

$$\text{score}^c(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i^c \cdot \mathbf{k}_j^c}{\sqrt{d_k}}$$

$$\alpha_{ij}^c = \text{softmax}(\text{score}^c(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \leq i$$

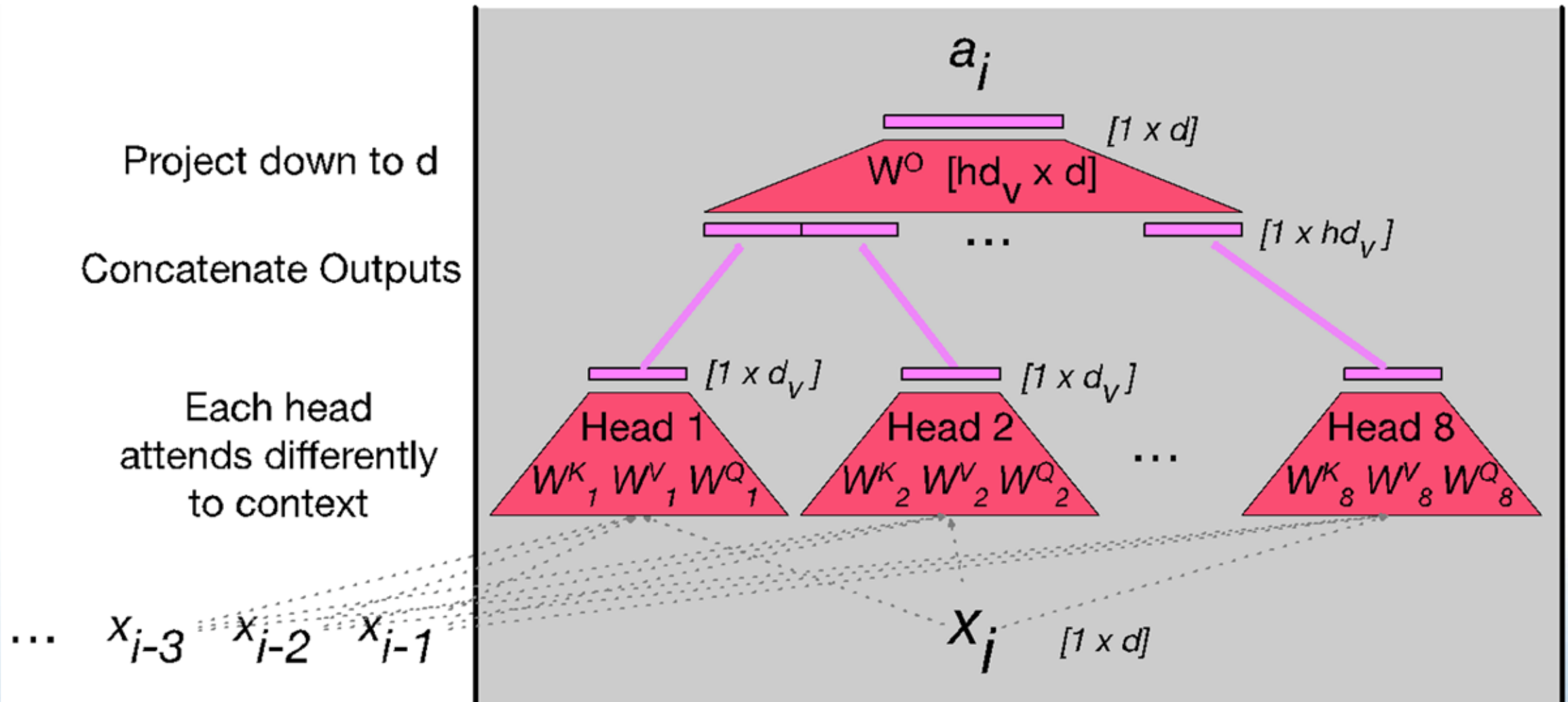
$$\text{head}_i^c = \sum_{j \leq i} \alpha_{ij}^c \mathbf{v}_j^c$$

$$\mathbf{a}_i = (\text{head}^1 \oplus \text{head}^2 \dots \oplus \text{head}^h) \mathbf{W}^O$$

$$\text{MultiHeadAttention}(\mathbf{x}_i, [\mathbf{x}_1, \dots, \mathbf{x}_N]) = \mathbf{a}_i$$



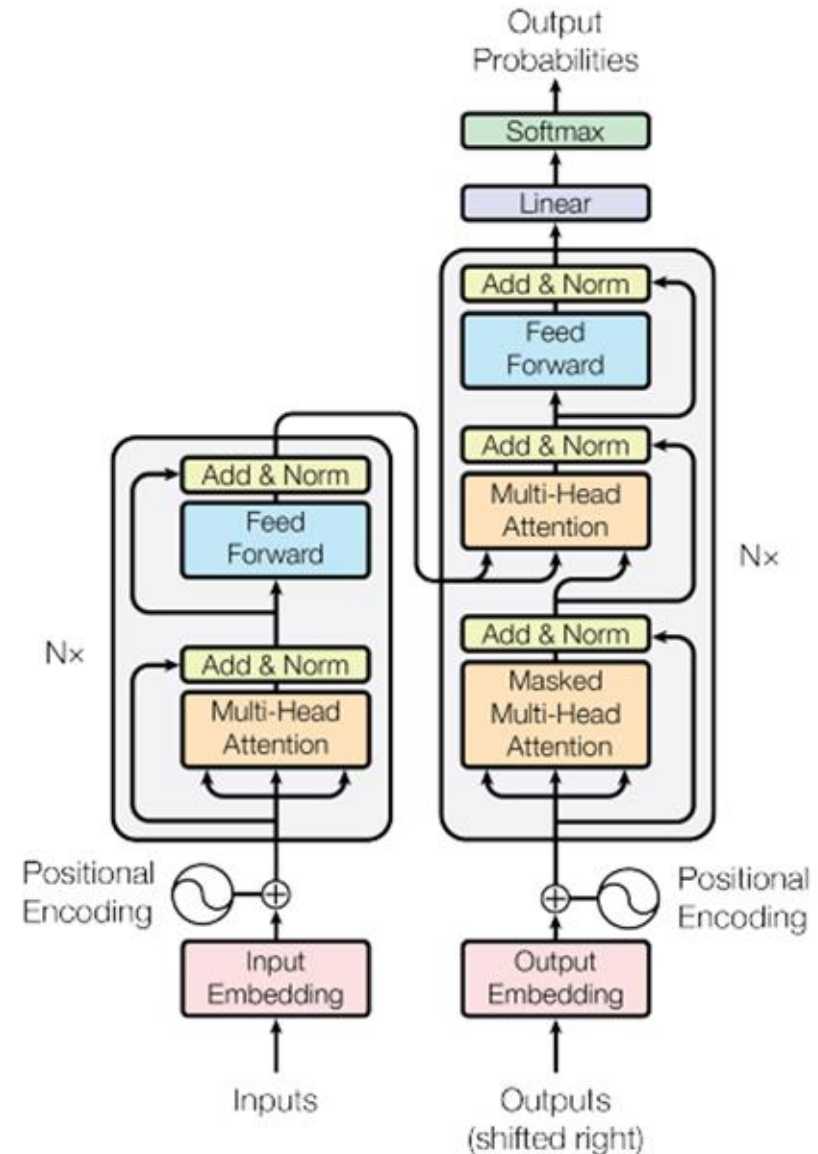
Multi-head Attention





Transformer

- [Attention is all you need](#)
- Encoder—Decoder architecture
- The Illustrated Transformer

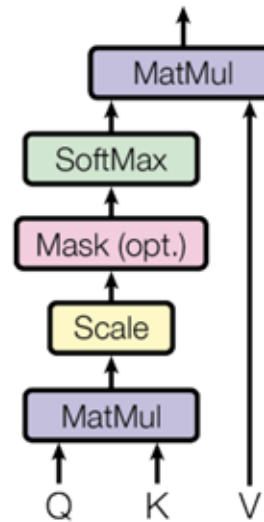




Multi-Head Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled Dot-Product Attention



Multi-Head Attention

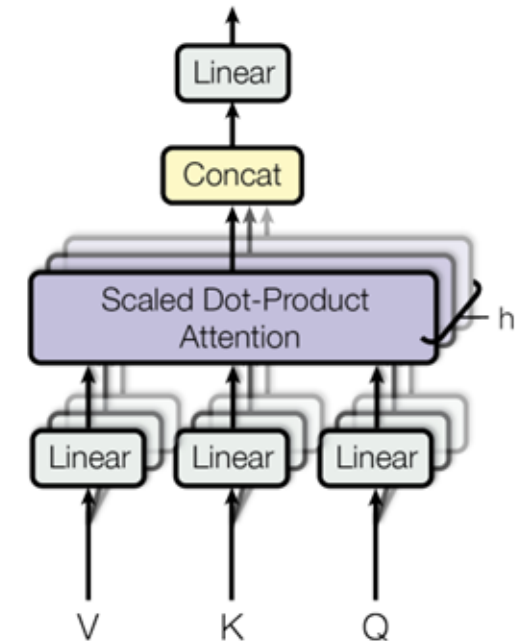


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.



Attention in Transformer models

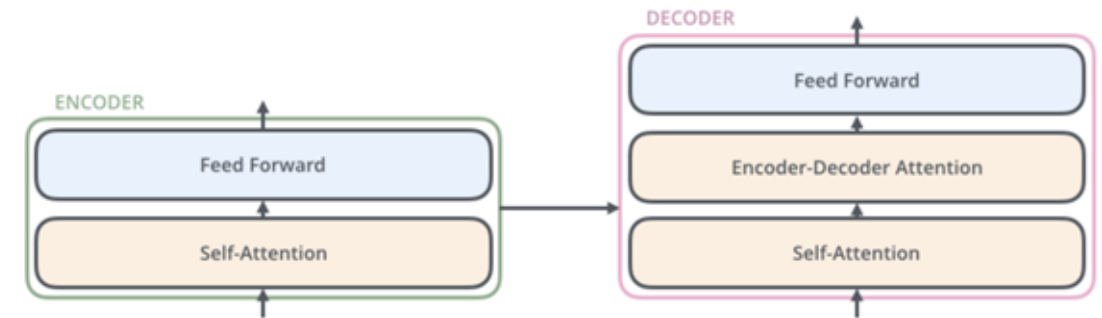
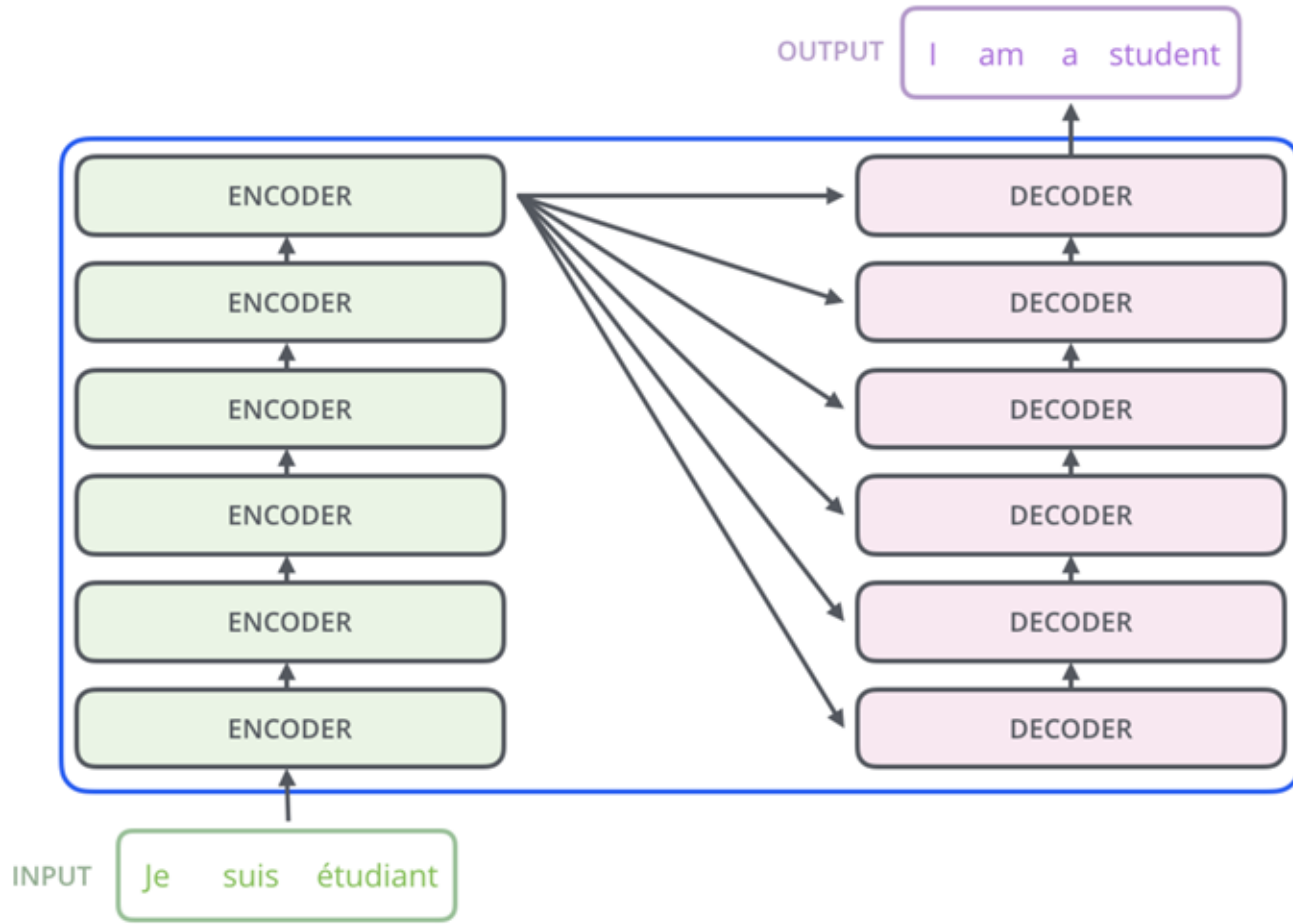
The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers,
 - the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder.
 - This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models.
- The encoder contains self-attention layers.
 - In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder.
 - Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder
 - allow each position in the decoder to attend to all positions in the decoder up to and including that position.
 - We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections.



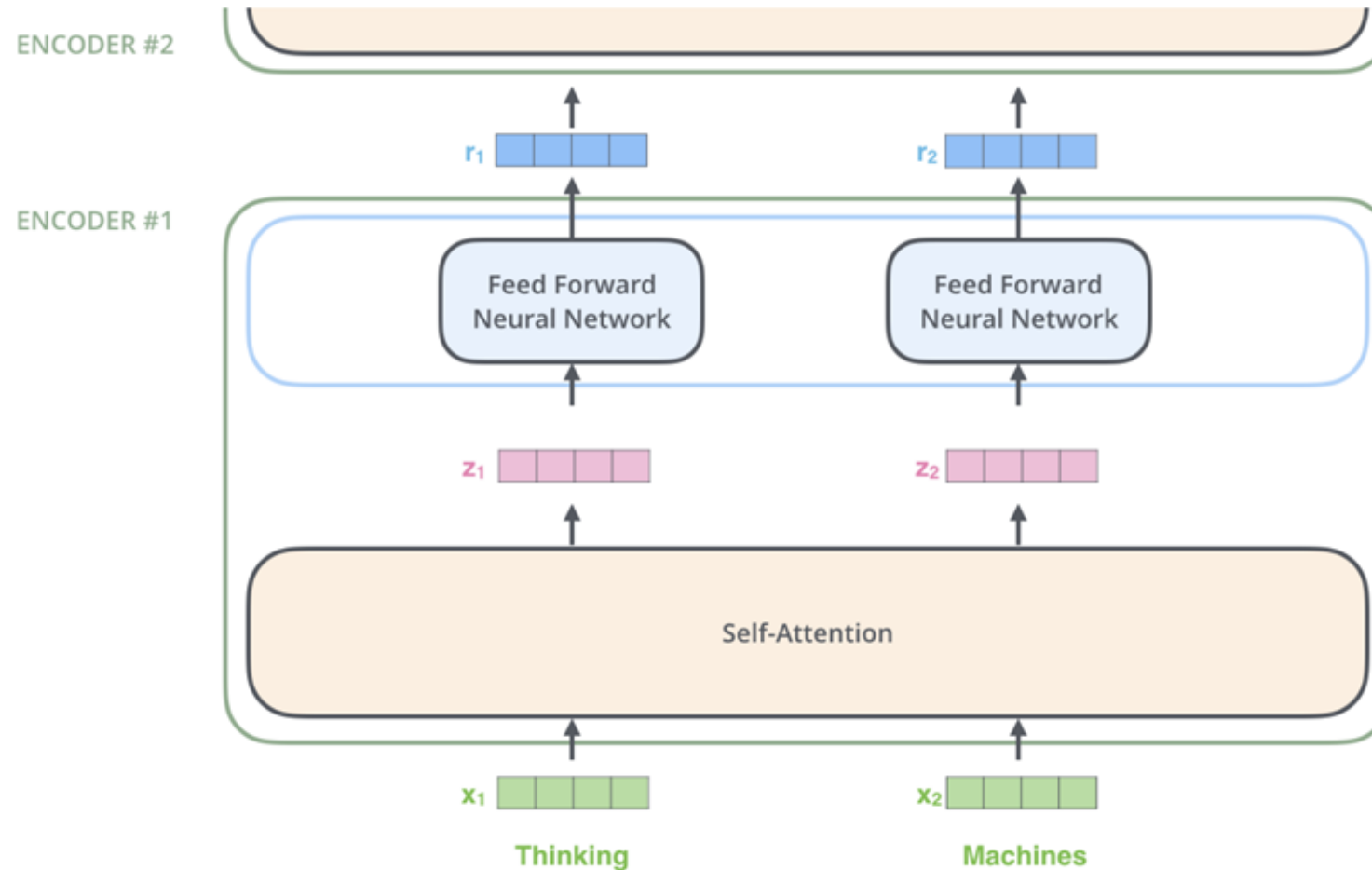
Transformer

– a parallelizable encoder-decoder architecture





Flow of Tensors in Transformers

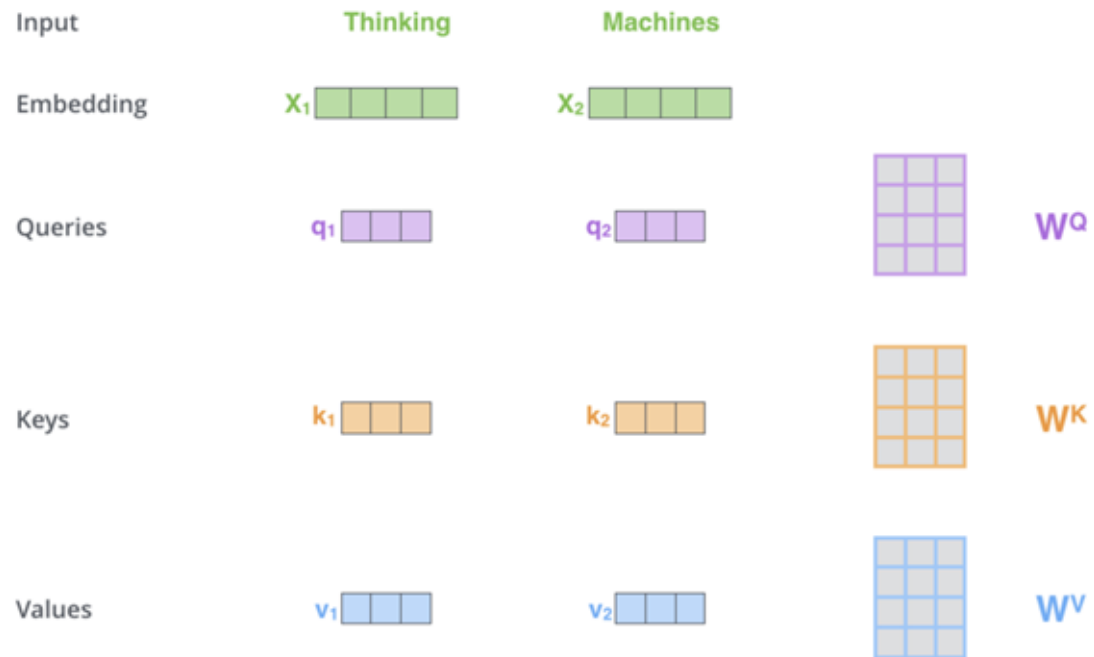


The word at each position passes through a self-attention process. Then, they each pass through a feed-forward neural network -- the exact same network with each vector flowing through it separately.



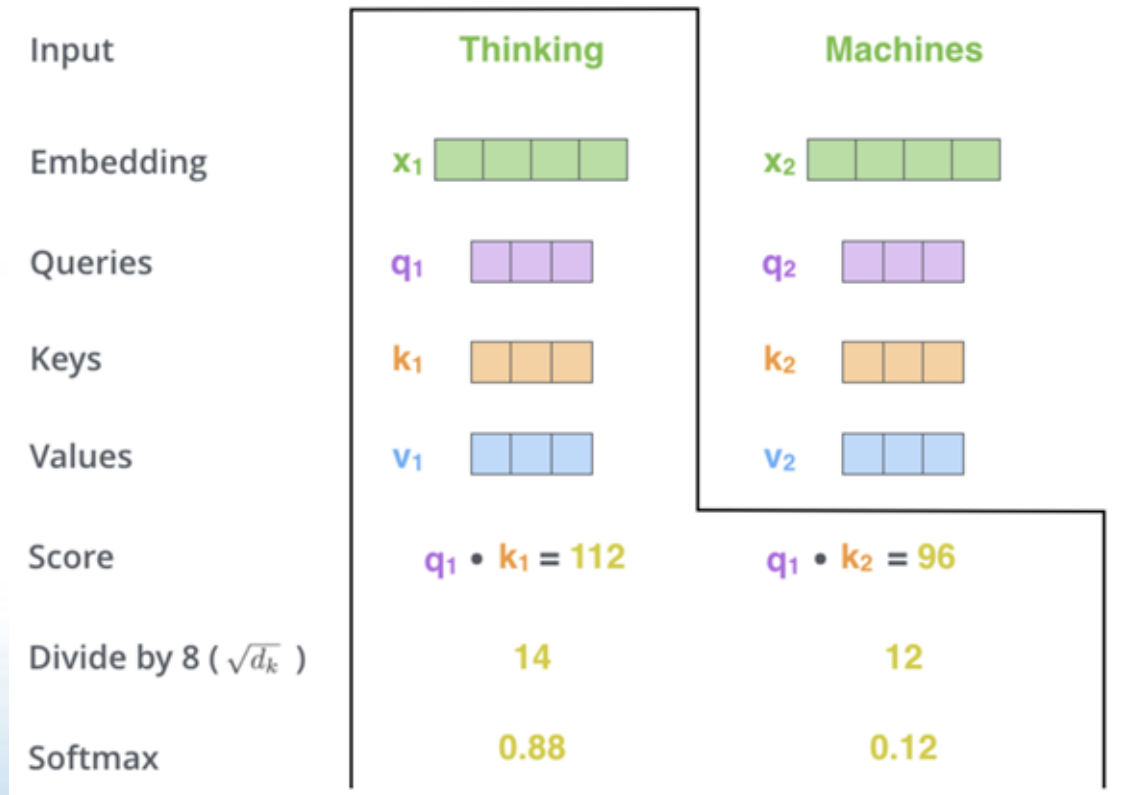
Self-Attention Steps

1. create three vectors (query, key, value) from each of the encoder's input vectors



Multiplying x_1 by the W_Q weight matrix produces q_1 , the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

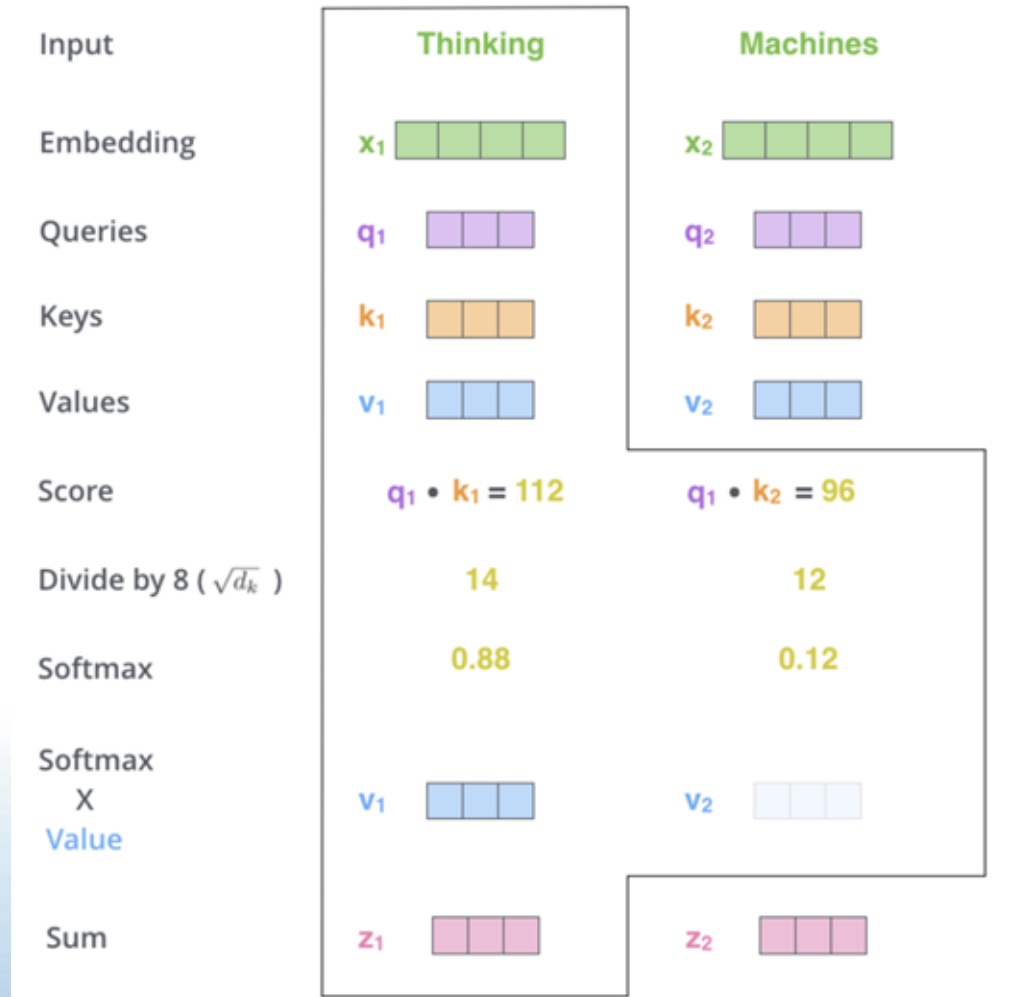
2. create a score for each input vector and pass normalized score through a softmax





Self-Attention Steps

3. multiply each value vector by the softmax score and sum the weighted value vectors





Matrix calculation of self-attention

$$\begin{matrix} X \\ \text{2x4 grid} \end{matrix} \times \begin{matrix} W^Q \\ \text{4x4 grid} \end{matrix} = \begin{matrix} Q \\ \text{2x3 grid} \end{matrix}$$

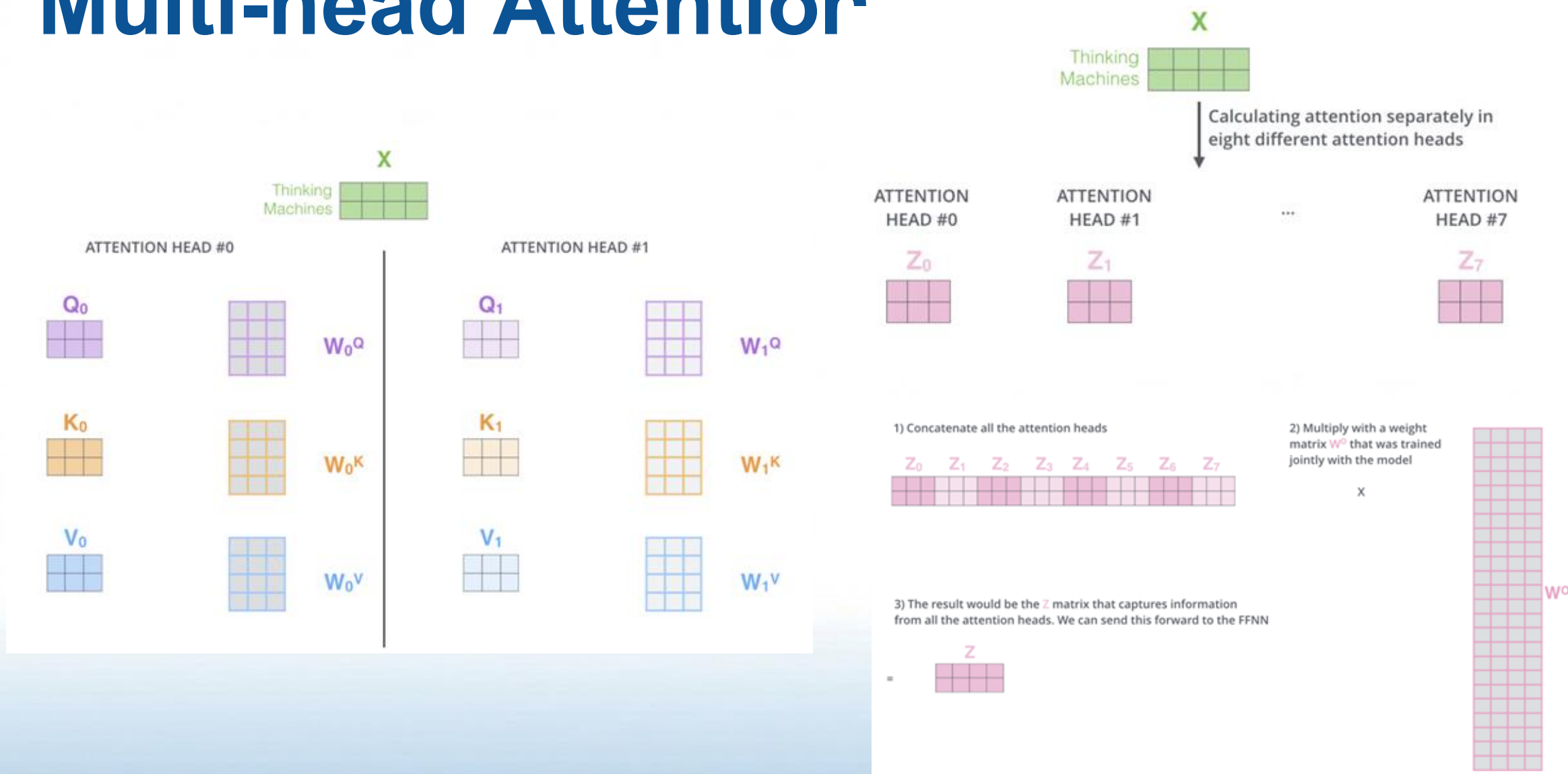
$$\begin{matrix} X \\ \text{2x4 grid} \end{matrix} \times \begin{matrix} W^K \\ \text{4x4 grid} \end{matrix} = \begin{matrix} K \\ \text{2x3 grid} \end{matrix}$$

$$\begin{matrix} X \\ \text{2x4 grid} \end{matrix} \times \begin{matrix} W^V \\ \text{4x4 grid} \end{matrix} = \begin{matrix} V \\ \text{2x3 grid} \end{matrix}$$

$$\text{softmax} \left(\frac{\begin{matrix} Q \\ \text{2x3 grid} \end{matrix} \times \begin{matrix} K^T \\ \text{3x3 grid} \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} V \\ \text{2x3 grid} \end{matrix}$$
$$= \begin{matrix} Z \\ \text{2x3 grid} \end{matrix}$$



Multi-head Attention

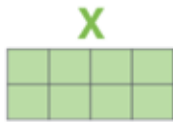




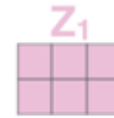
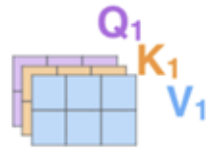
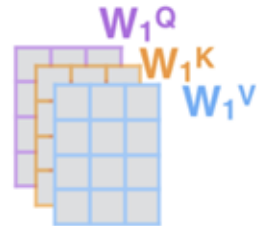
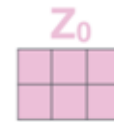
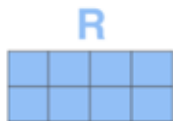
Self attention summary

- 1) This is our input sentence*
- 2) We embed each word*
- 3) Split into 8 heads. We multiply X or R with weight matrices
- 4) Calculate attention using the resulting $Q/K/V$ matrices
- 5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Thinking
Machines



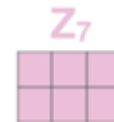
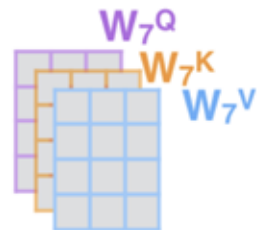
* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



...

...

...



Transformers

Parallelizing Attention Computation



Parallelizing computation using X

- For attention/transformer block we've been computing a **single** output at a **single** time step i in a **single** residual stream.
- But we can pack the N tokens of the input sequence into a single matrix X of size $[N \times d]$.
- Each row of X is the embedding of one token of the input.
- X can have 1K-128K rows, each of the dimensionality of the embedding d (the model dimension)

$$Q = XW^Q, K = XW^K, V = XW^V$$



QK^T

Now can do a single matrix multiply to combine Q and K^T

N	$q1 \cdot k1$	$q1 \cdot k2$	$q1 \cdot k3$	$q1 \cdot k4$
	$q2 \cdot k1$	$q2 \cdot k2$	$q2 \cdot k3$	$q2 \cdot k4$
	$q3 \cdot k1$	$q3 \cdot k2$	$q3 \cdot k3$	$q3 \cdot k4$
	$q4 \cdot k1$	$q4 \cdot k2$	$q4 \cdot k3$	$q4 \cdot k4$
	N			



Parallelizing attention

- Scale the scores, take the softmax, and then multiply the result by V resulting in a matrix of shape $N \times d$
 - An attention vector for each input token

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$



Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

- What is this mask function?
 $\mathbf{Q}\mathbf{K}^\top$ has a score for each query dot every key, *including those that follow the query.*
- Guessing the next word is pretty simple if you already know it!



Masking out the future

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{V}$$

Add $-\infty$ to cells in upper triangle
The softmax will turn it to 0

N

q1·k1	$-\infty$	$-\infty$	$-\infty$
q2·k1	q2·k2	$-\infty$	$-\infty$
q3·k1	q3·k2	q3·k3	$-\infty$
q4·k1	q4·k2	q4·k3	q4·k4

N



Another point: Attention is quadratic in length

$$\mathbf{A} = \text{softmax} \left(\text{mask} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} \right) \right) \mathbf{v}$$

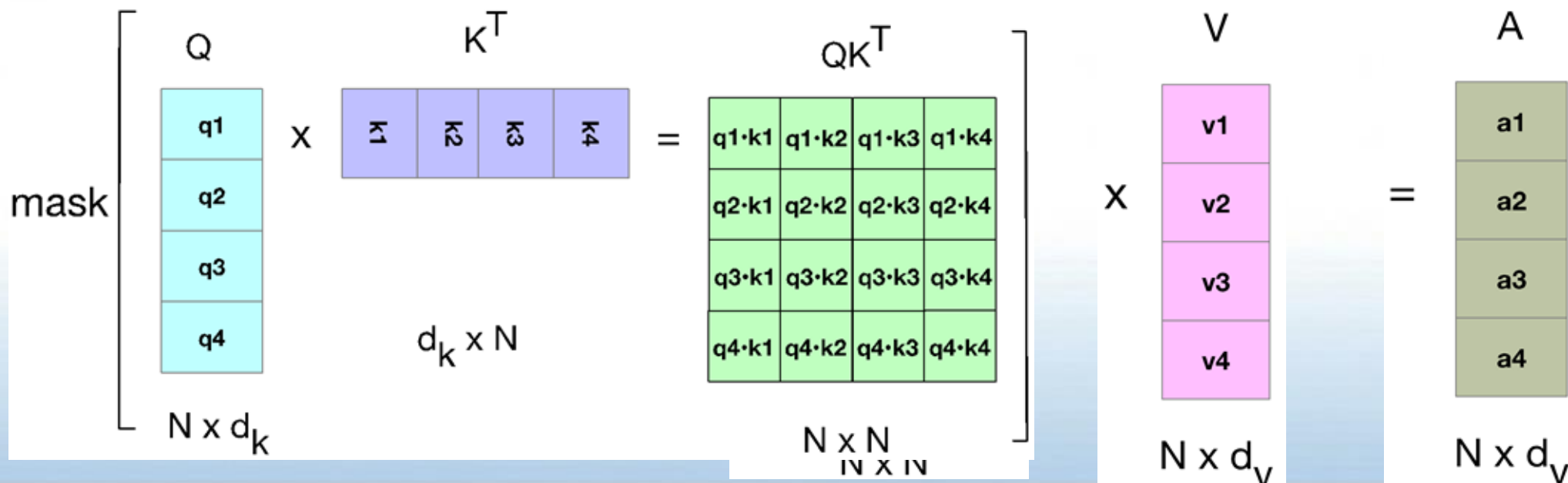
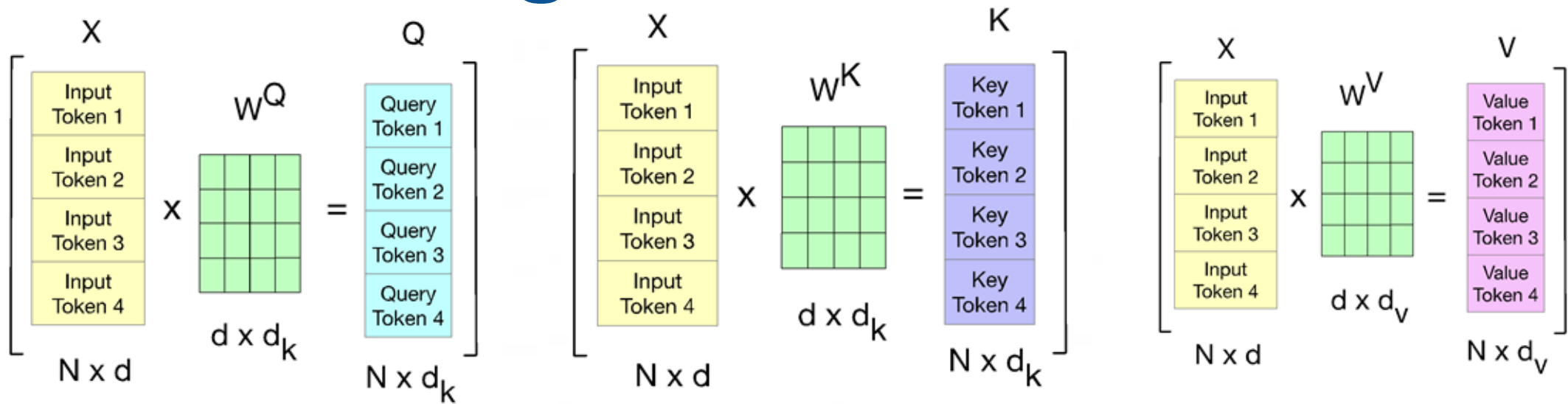
N

q1•k1	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞
q4•k1	q4•k2	q4•k3	q4•k4

N



Attention again





Parallelizing Multi-head Attention

$$\mathbf{Q}^i = \mathbf{XW}^{\mathbf{Q}^i}; \quad \mathbf{K}^i = \mathbf{XW}^{\mathbf{K}^i}; \quad \mathbf{V}^i = \mathbf{XW}^{\mathbf{V}^i}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}^i, \mathbf{K}^i, \mathbf{V}^i) = \text{softmax}\left(\frac{\mathbf{Q}^i \mathbf{K}^{i\top}}{\sqrt{d_k}}\right) \mathbf{V}^i$$

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \dots \oplus \mathbf{head}_h) \mathbf{W}^{\mathbf{C}}$$



Parallelizing Multi-head Attention

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{MultiHeadAttention}(\mathbf{X}))$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O}))$$

or

$$\mathbf{T}^1 = \text{MultiHeadAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}^3 = \text{LayerNorm}(\mathbf{T}^2)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

Transformers

*Input and output:
Position embeddings and the
Language Model Head*



Token and Position Embeddings

The matrix X (of shape $[N \times d]$) has an embedding for each word in the context.

This embedding is created by adding two distinct embedding for each input

- token embedding
- positional embedding



Token Embeddings

Embedding matrix E has shape $[|V| \times d]$.

- One row for each of the $|V|$ tokens in the vocabulary.
- Each word is a row vector of d dimensions

Given: string "*Thanks for all the*"

1. Tokenize with BPE and convert into vocab indices

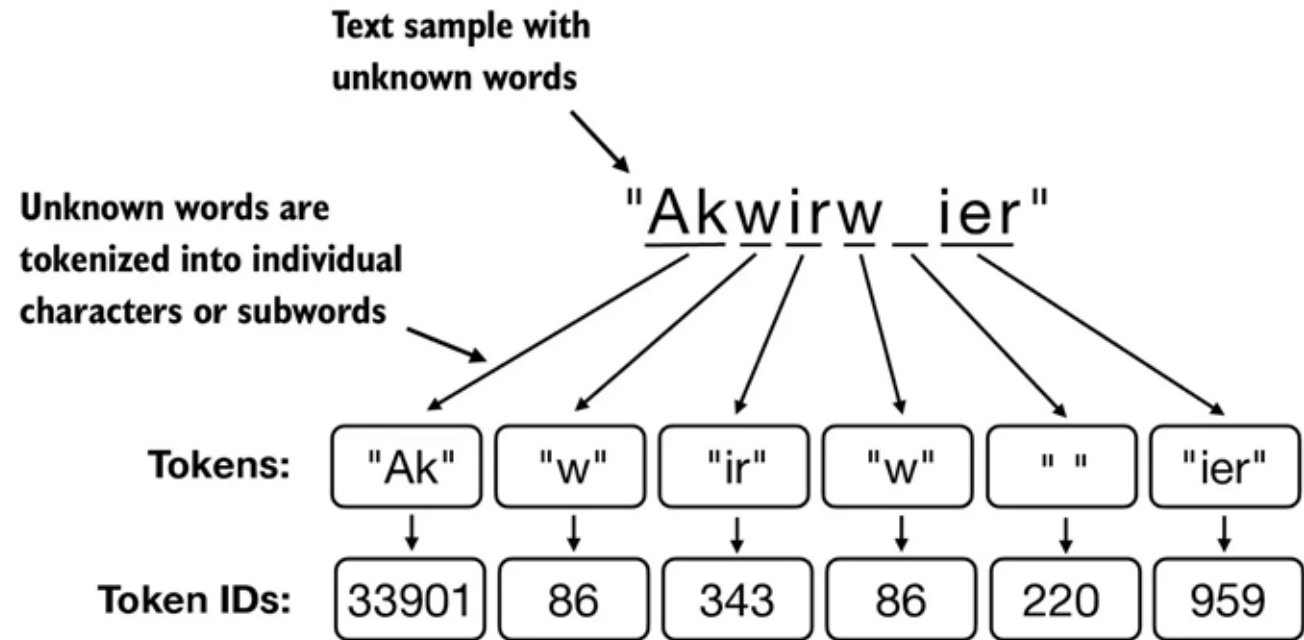
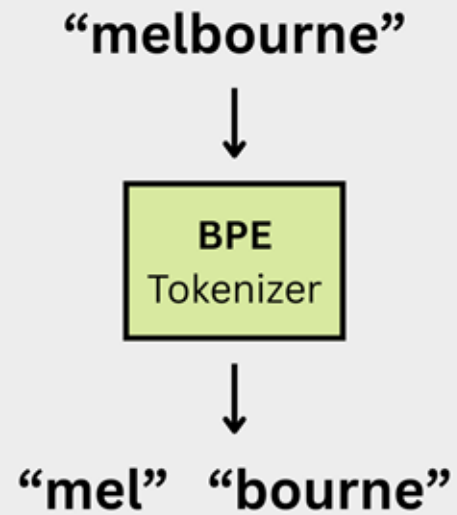
$w = [5, 4000, 10532, 2224]$

2. Select the corresponding rows from E , each row an embedding

- (row 5, row 4000, row 10532, row 2224).



BPE example





Position Embeddings

There are many methods, but we'll just describe the simplest: absolute position.

Goal: learn a position embedding matrix E_{pos} of shape $[1 \times N]$.

Start with randomly initialized embeddings

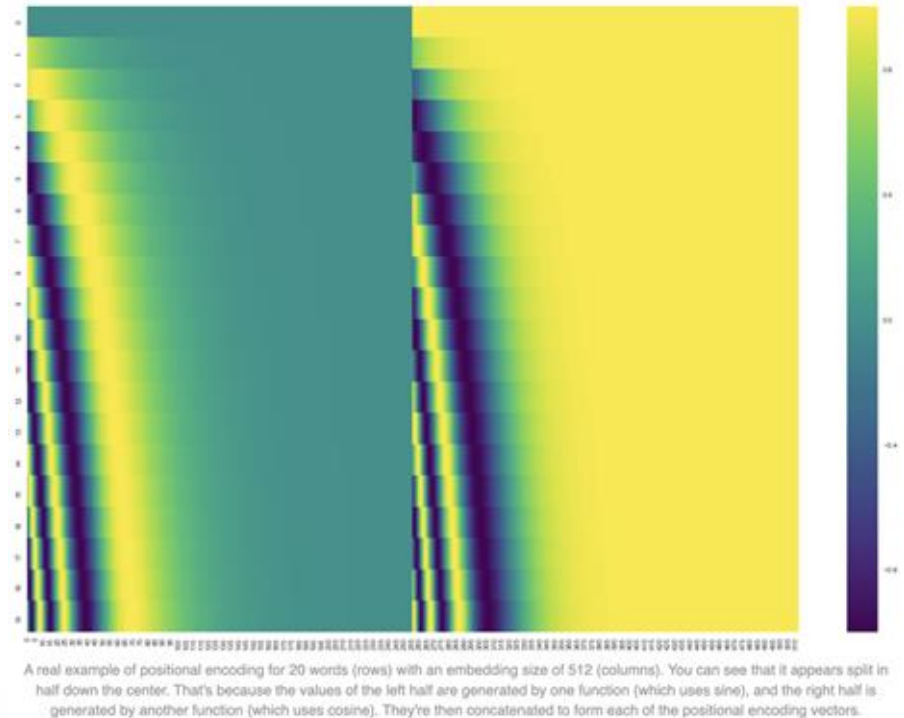
- one for each integer up to some maximum length.
- i.e., just as we have an embedding for token *fish*, we'll have an embedding for position 3 and position 17.
- Position embedding can be fixed or learned/trainable
- As with word embeddings, these position embeddings can be learned along with other parameters during training.



Position embedding

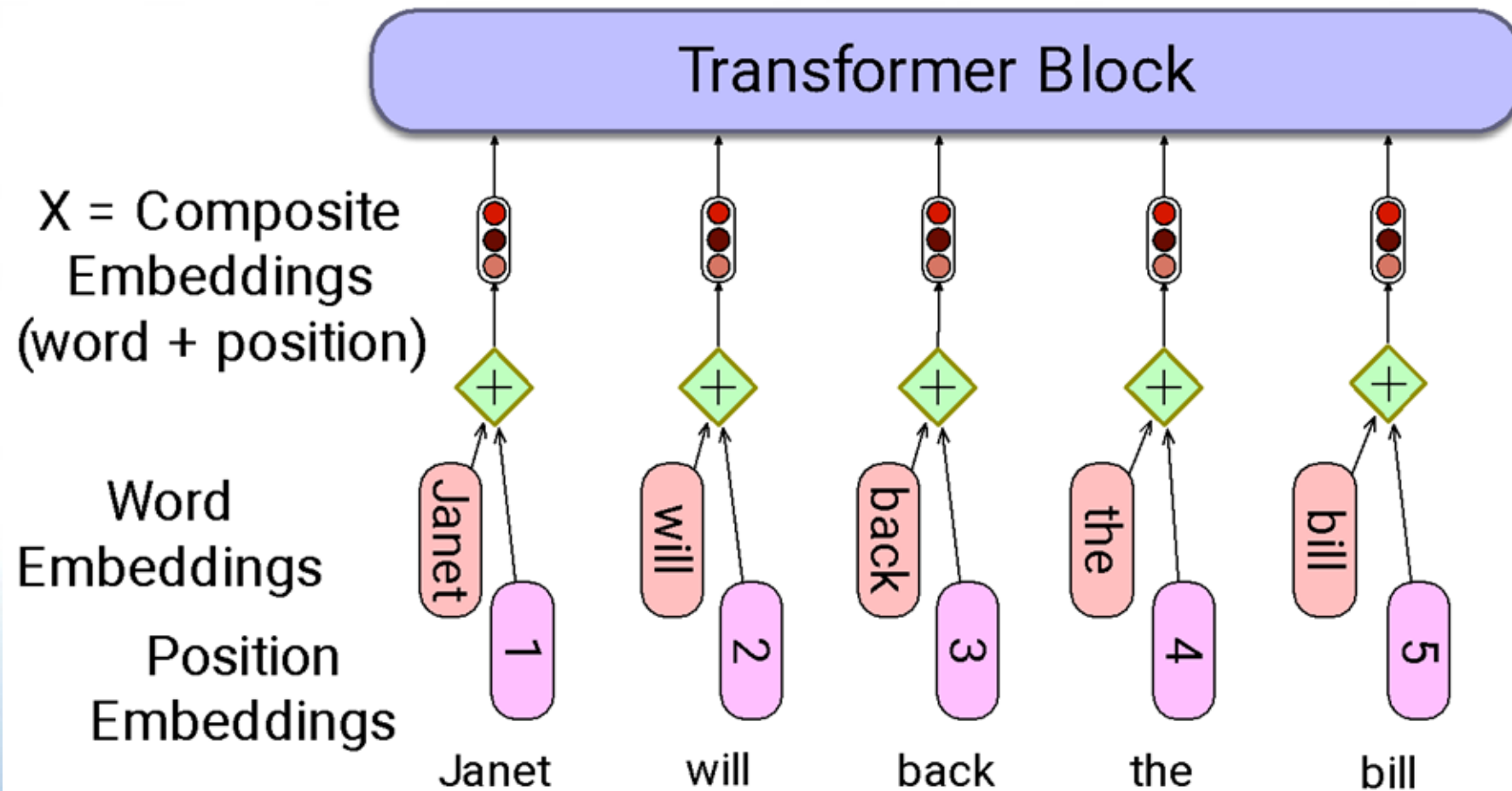
- Self-attention is permutation invariant
- Self-attention is a set operation not sequence
- Without position information in the input embedding
 - Transformer block embedding for "cat sat on mat" will be same as "mat on sat cat"
- Position encoding breaks this symmetry
- Sinusoidal embedding (original Transformer paper)
- Write the equation:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$$





Each x is just the sum of word and position embeddings

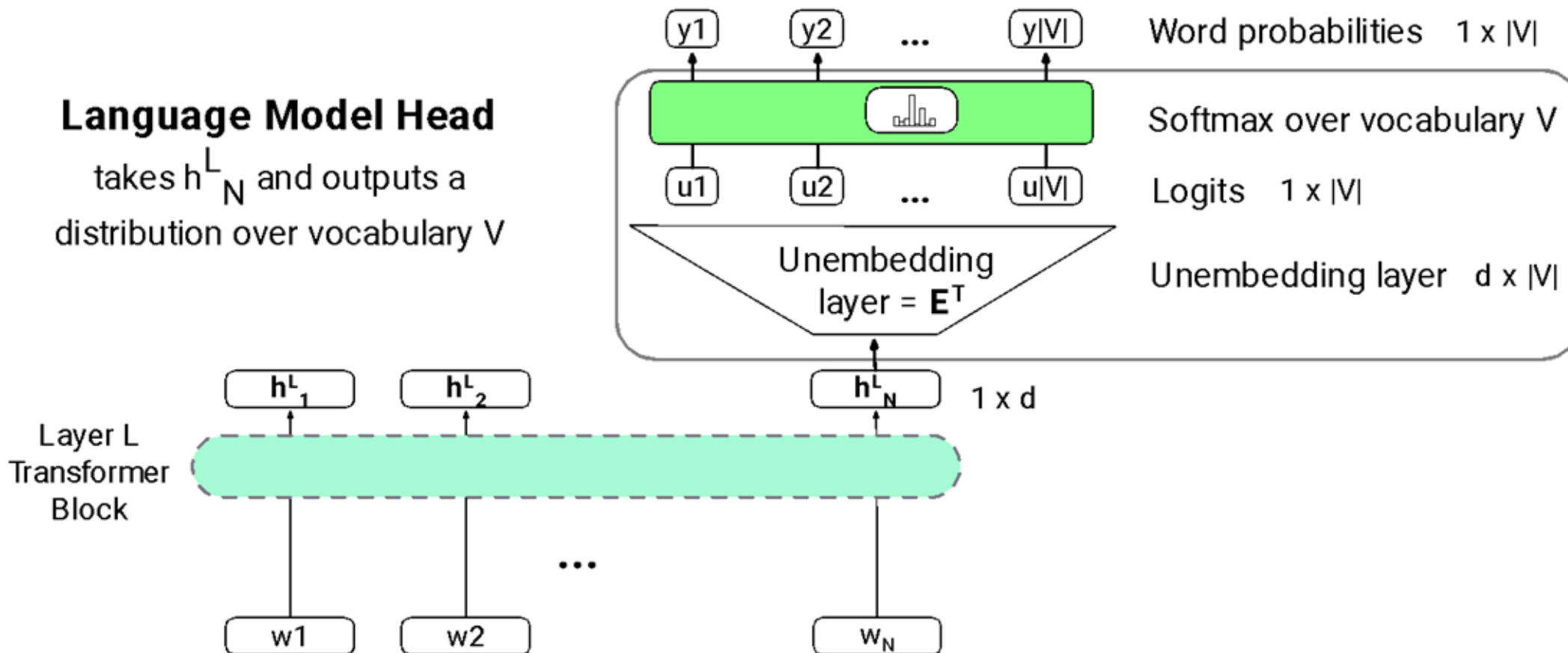




Language modeling head

Language Model Head

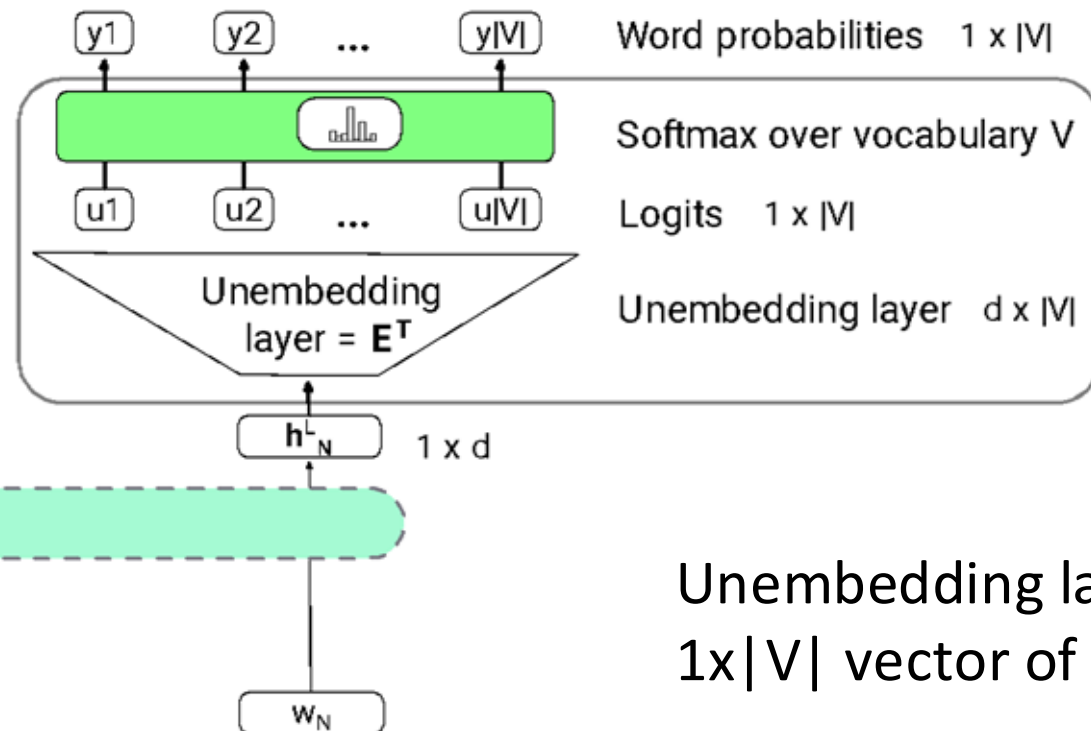
takes h_N^L and outputs a distribution over vocabulary V





Language modeling head

Unembedding layer: linear layer projects from h_N^L (shape $[1 \times d]$) to logit vector



Why "unembedding"? **Tied** to E^T

Weight tying, we use the same weights for two different matrices

Unembedding layer maps from an embedding to a $1 \times |V|$ vector of logits



Why weight tying?

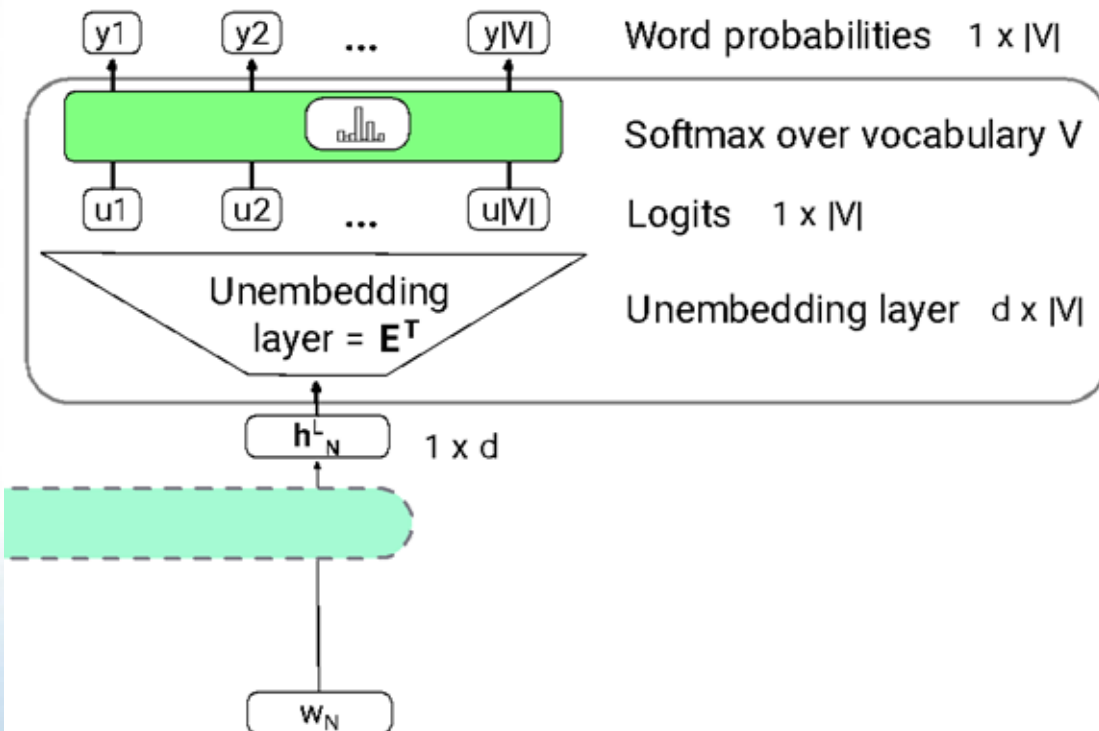
- Standard LLM pipeline: Input tokens $\rightarrow$ Embedding Layer $\rightarrow$ Transformer Layers $\rightarrow$ Unembedding Layer $\rightarrow$ Logits $\rightarrow$ Softmax
- **Embedding Layer:** Maps token IDs to dense vectors
 $\mathbf{E} \in \mathbb{R}^{(V \times d)}$: vocabulary size V , embedding dimension d
Input: token ID $\rightarrow$ Output: embedding vector
- **Unembedding Layer:** Maps hidden states to vocabulary logits
 $\mathbf{U} \in \mathbb{R}^{(d \times V)}$: projects d -dimensional hidden states to V logits
Input: hidden vector $\rightarrow$ Output: vocabulary logits
- Parameter reduction: 50%
- Improved generalization due to regularization



Language modeling head

Logits, the score vector u

One score for each of the $|V|$ possible words in the vocabulary V .
Shape $1 \times |V|$.



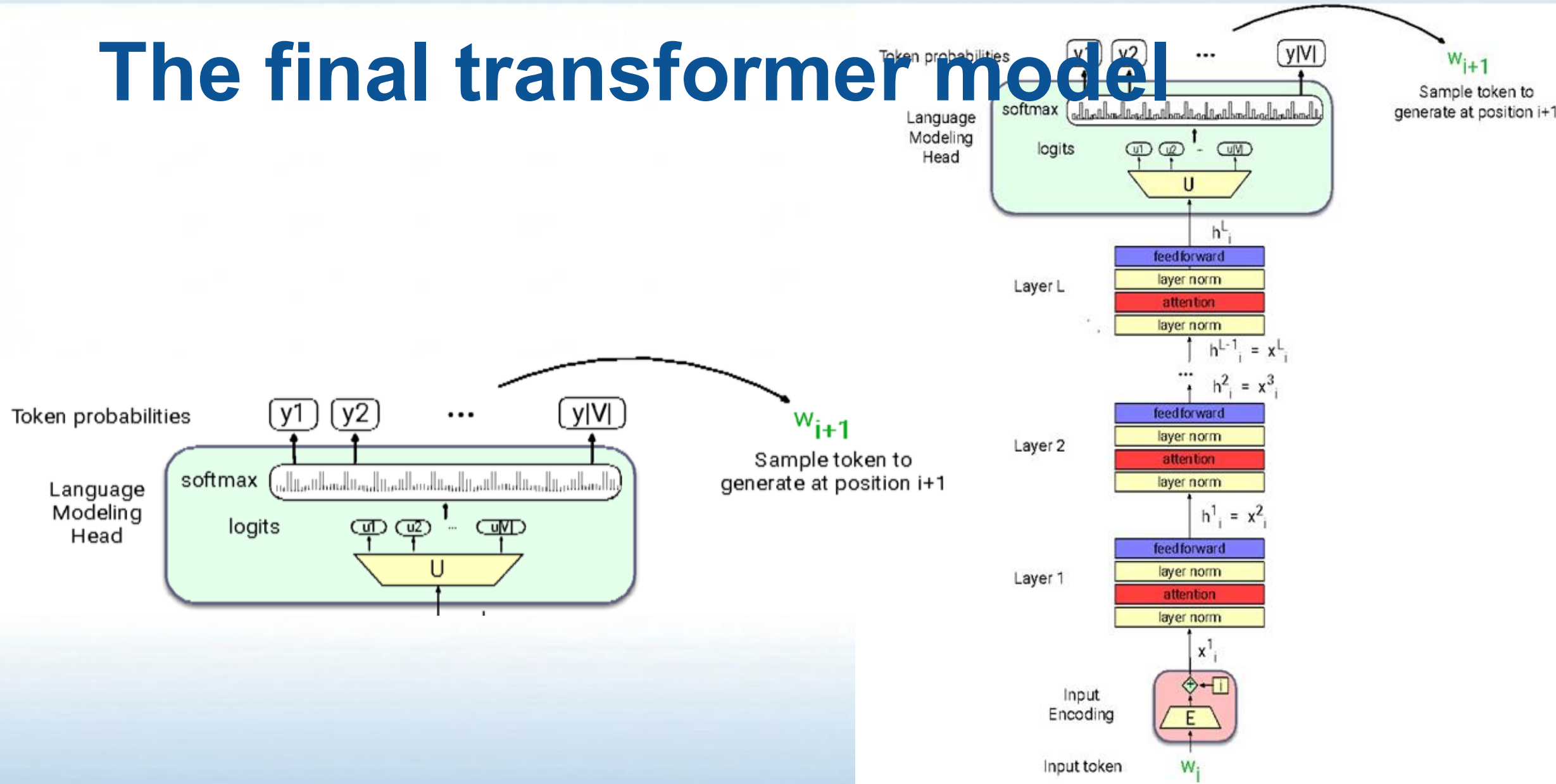
Softmax turns the logits into probabilities over vocabulary.
Shape $1 \times |V|$.

$$u = h_N^L E^T$$

$$y = \text{softmax}(u)$$

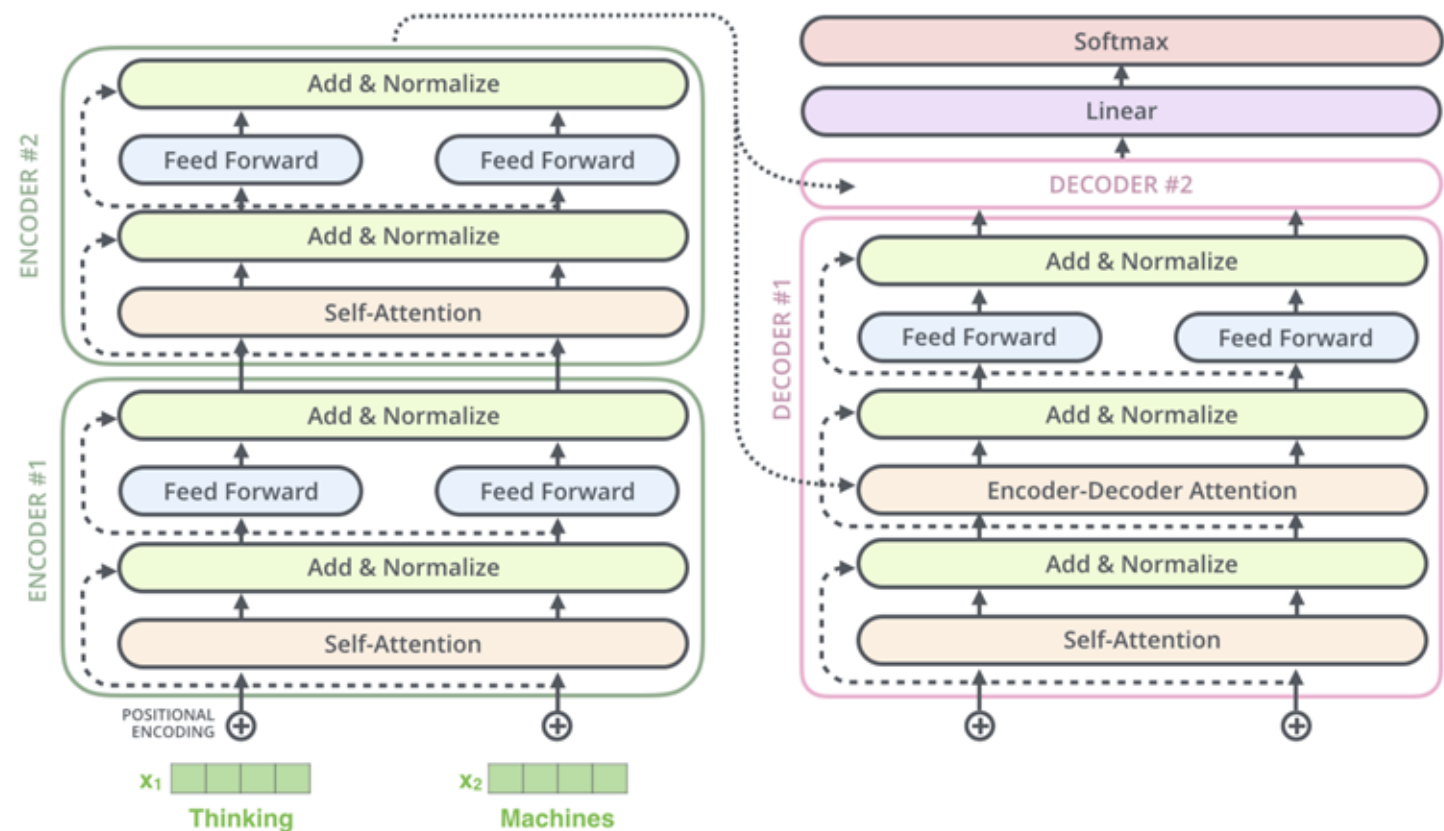
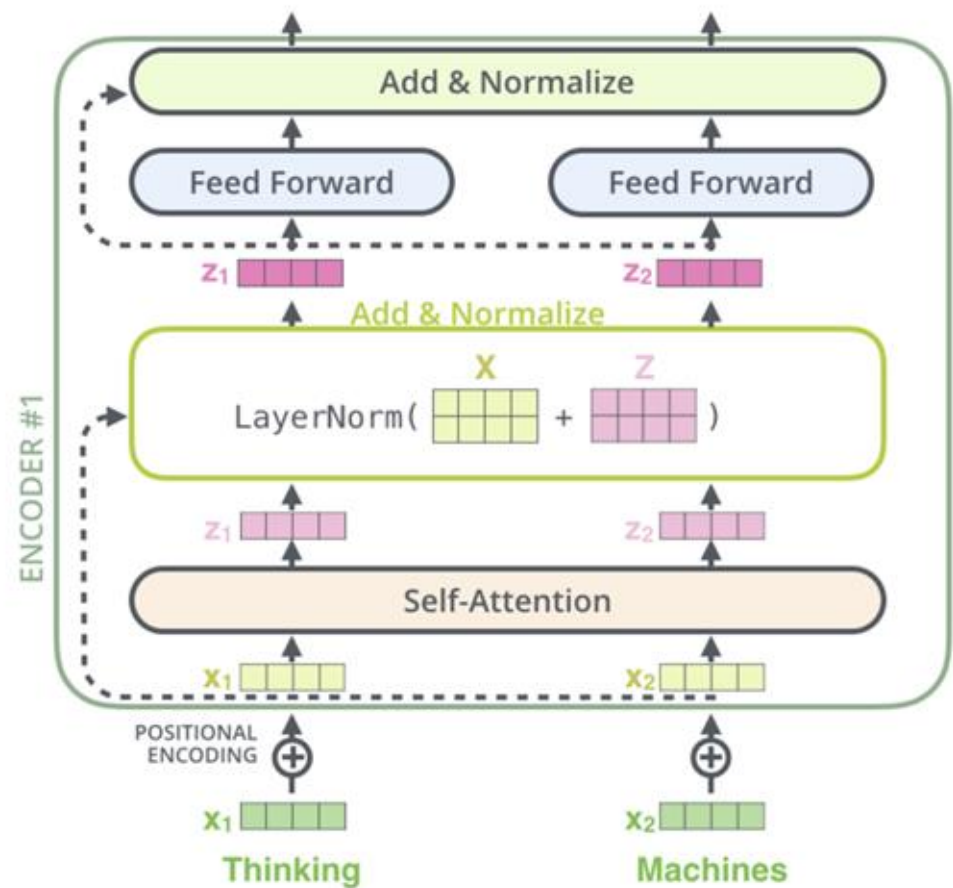


The final transformer model





Residual connection





Layer Norm

- Layer norm is a variation of the z-score from statistics, applied to a single vector in a hidden layer

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i$$

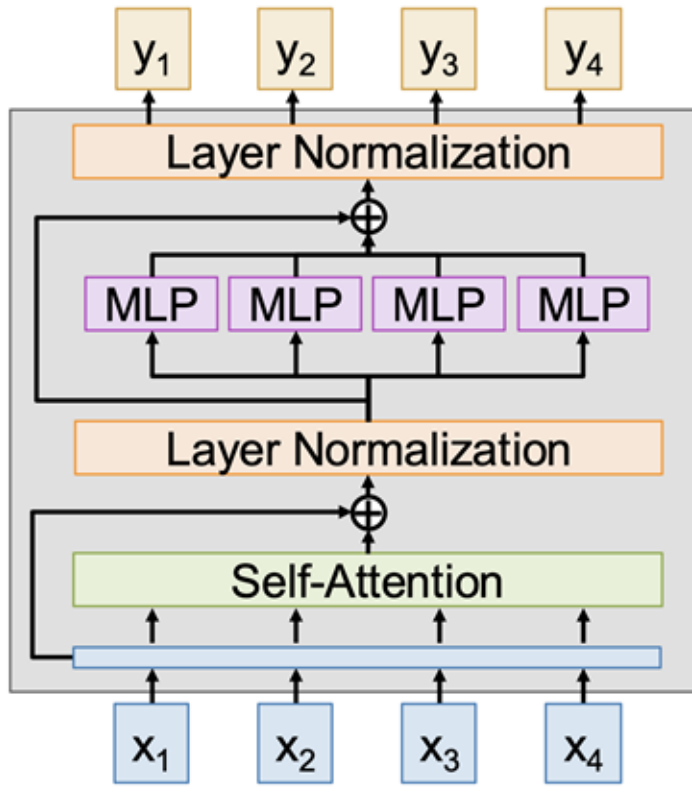
$$\sigma = \sqrt{\frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

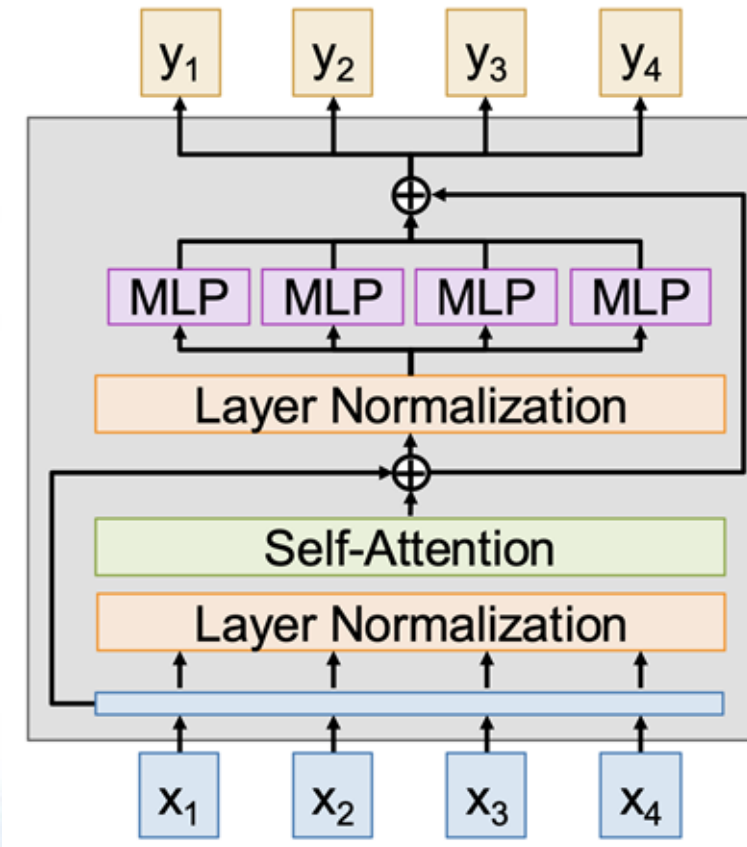
$$\text{LayerNorm}(\mathbf{x}) = \gamma \frac{(\mathbf{x} - \mu)}{\sigma} + \beta$$



Where to put normalization layer?



Post norm



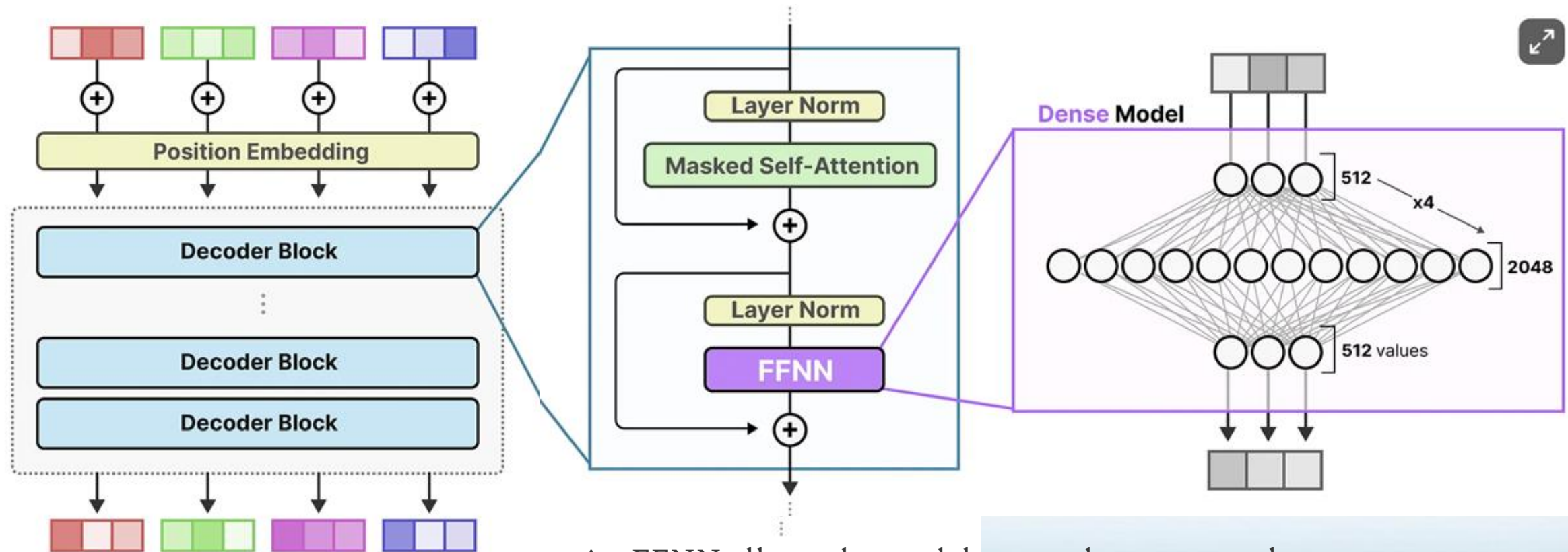
Pre-norm

Pre-norm results in

- **Superior gradient flow** via direct residual paths
- **Training stability** from initialization to convergence



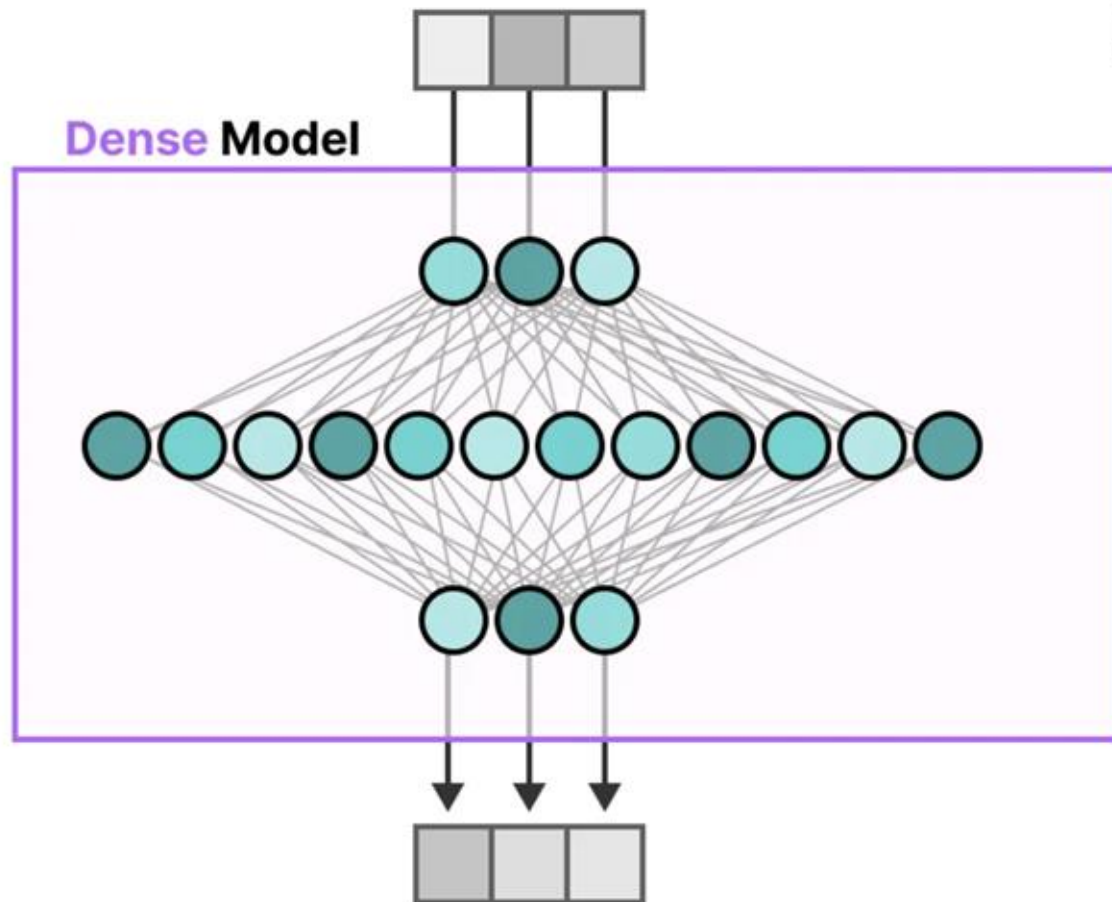
FFNN in Decoder



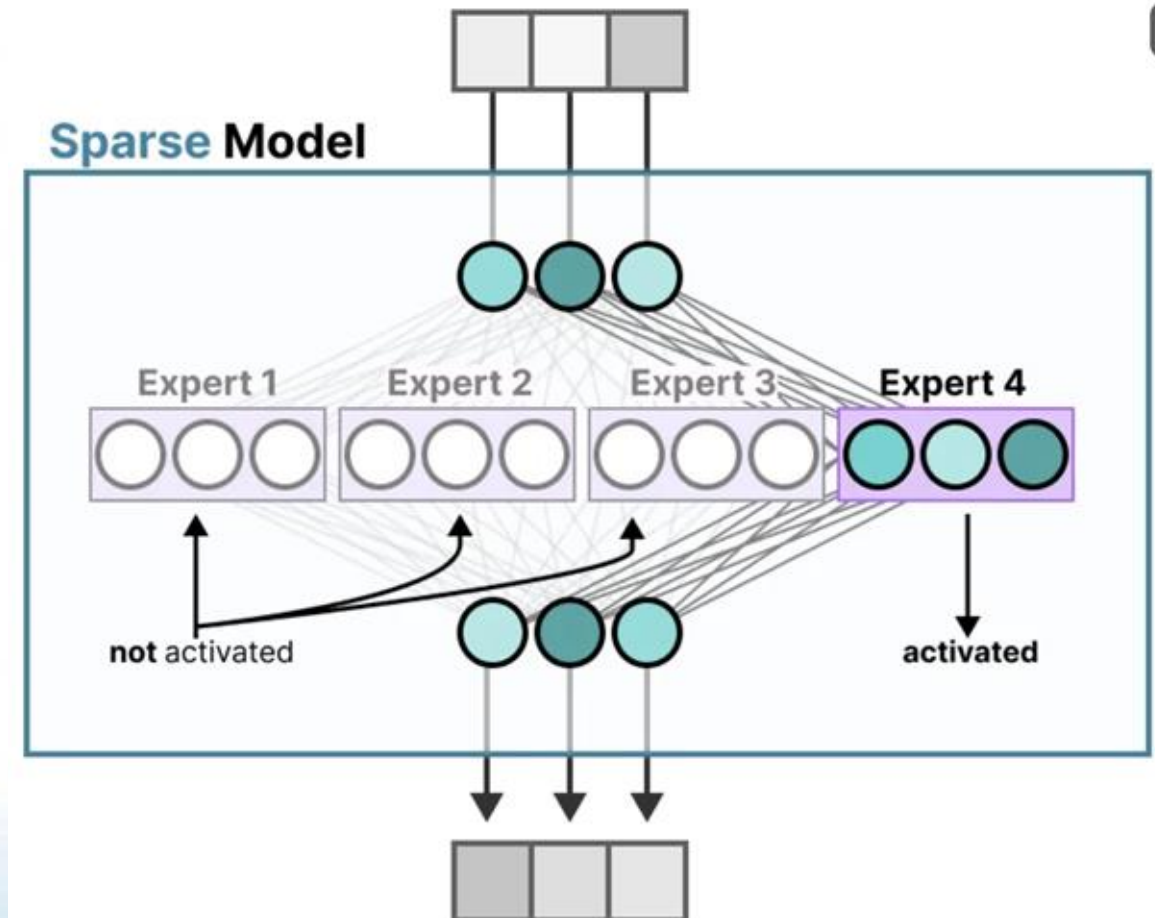
An FFNN allows the model to use the contextual information created by the attention mechanism, transforming it further to capture more complex relationships in the data



Dense vs Sparse Models



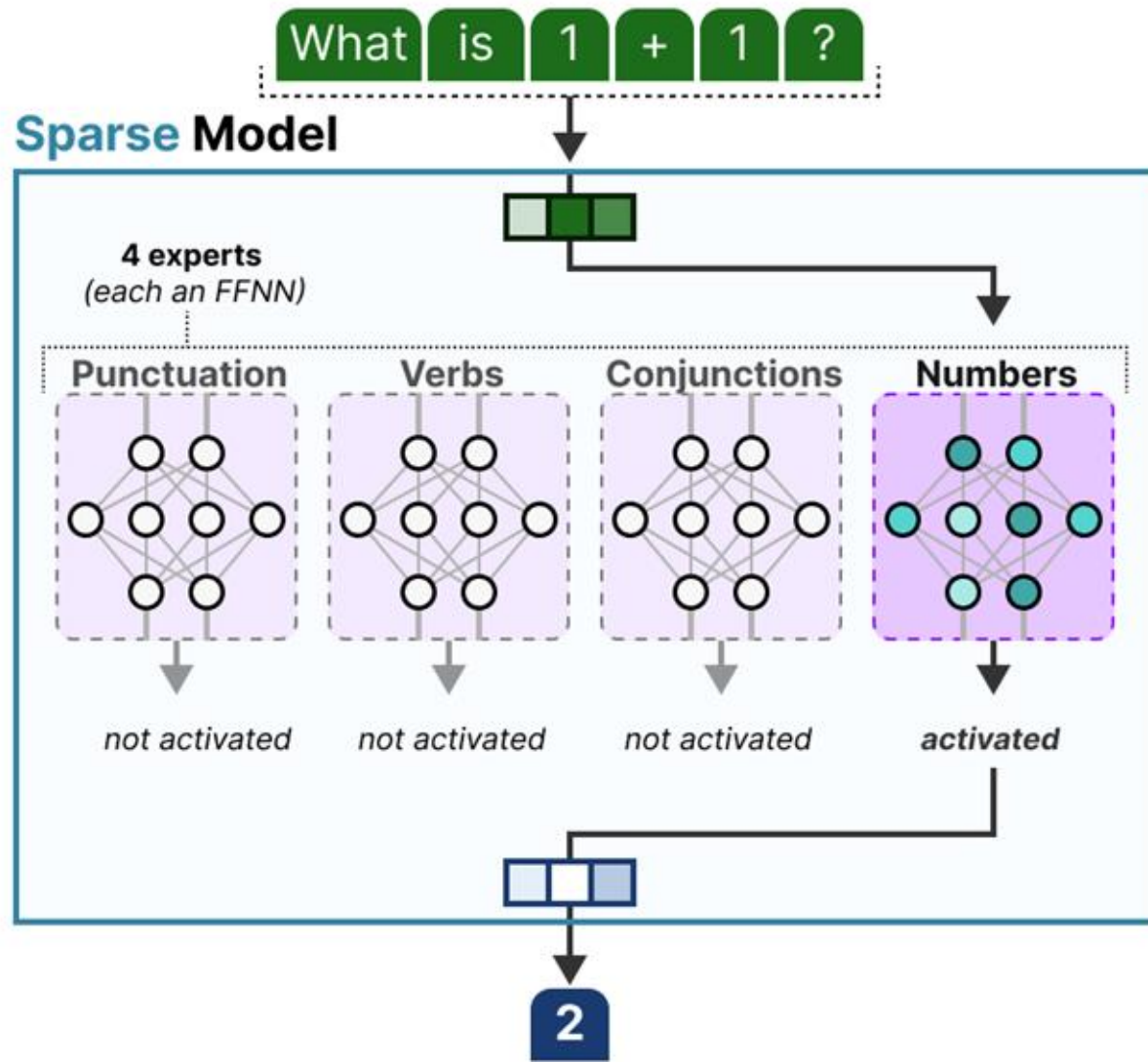
All parameters are activated for any input



Only a subset of parameters are activated for any input



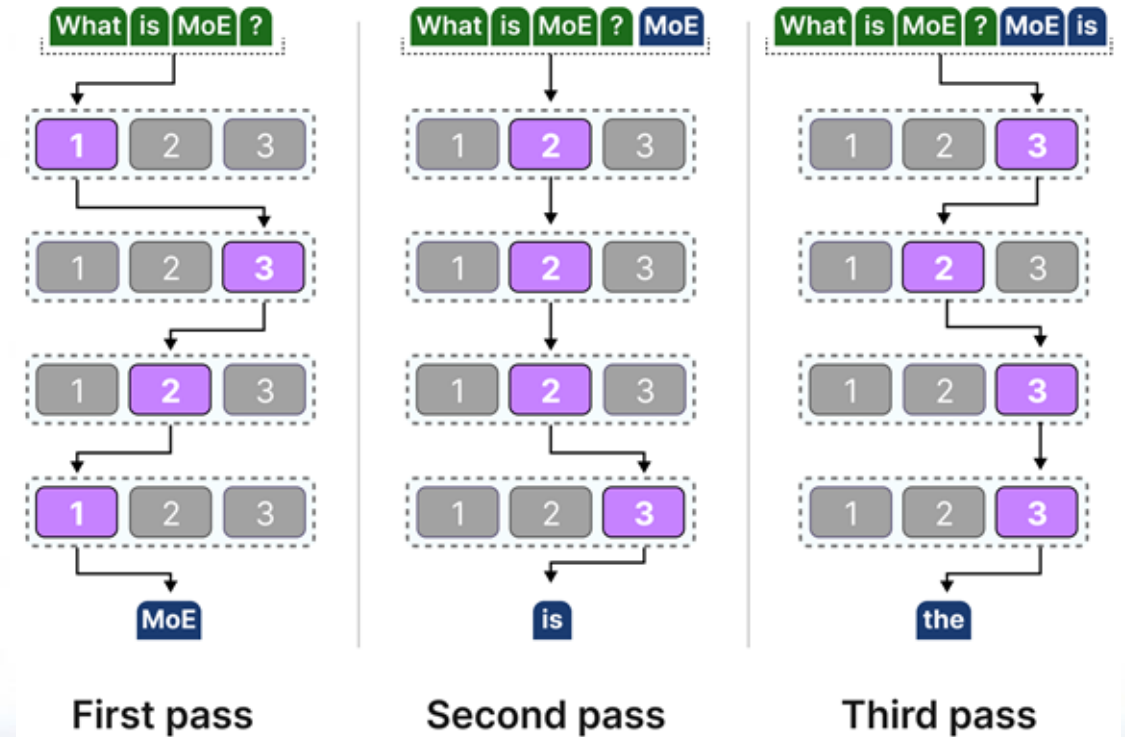
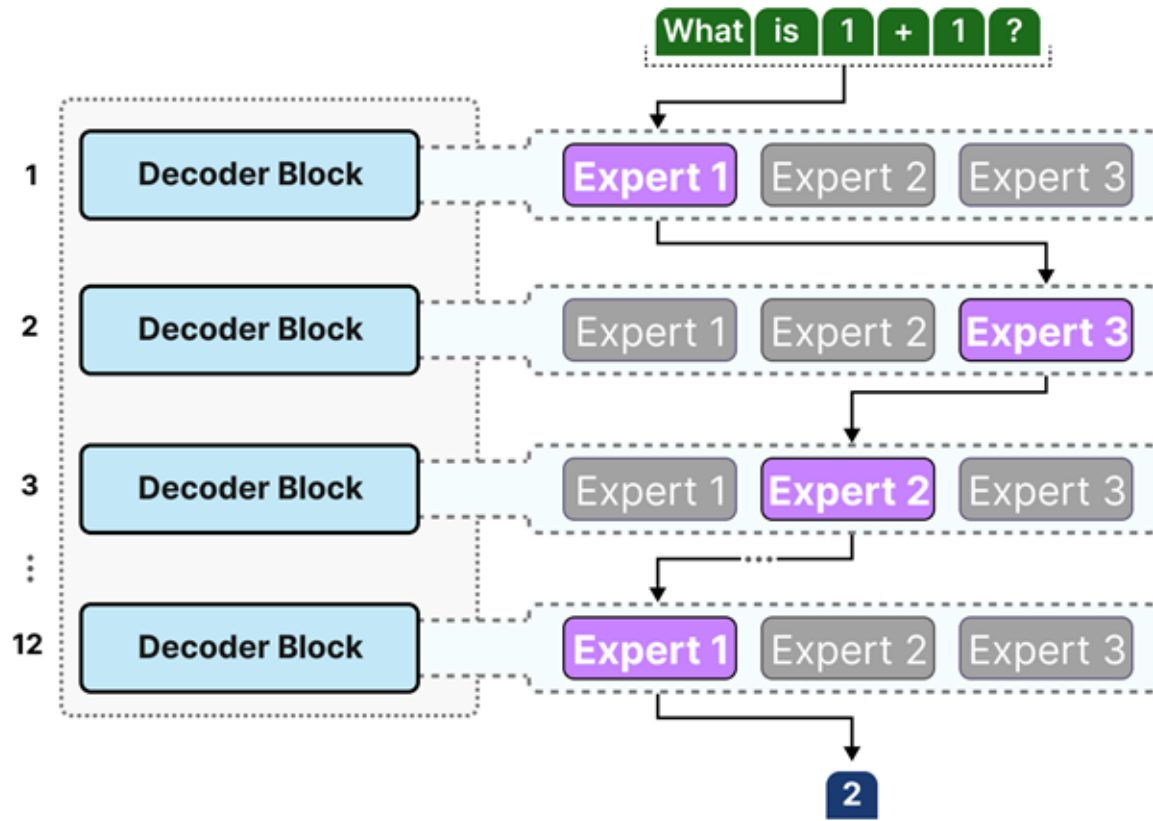
Architecture of Experts



- each expert learns different information during training
- during inference, only specific experts are used as they are most relevant for a given task
- each expert is often whole FFNNs themselves



Choice of Experts

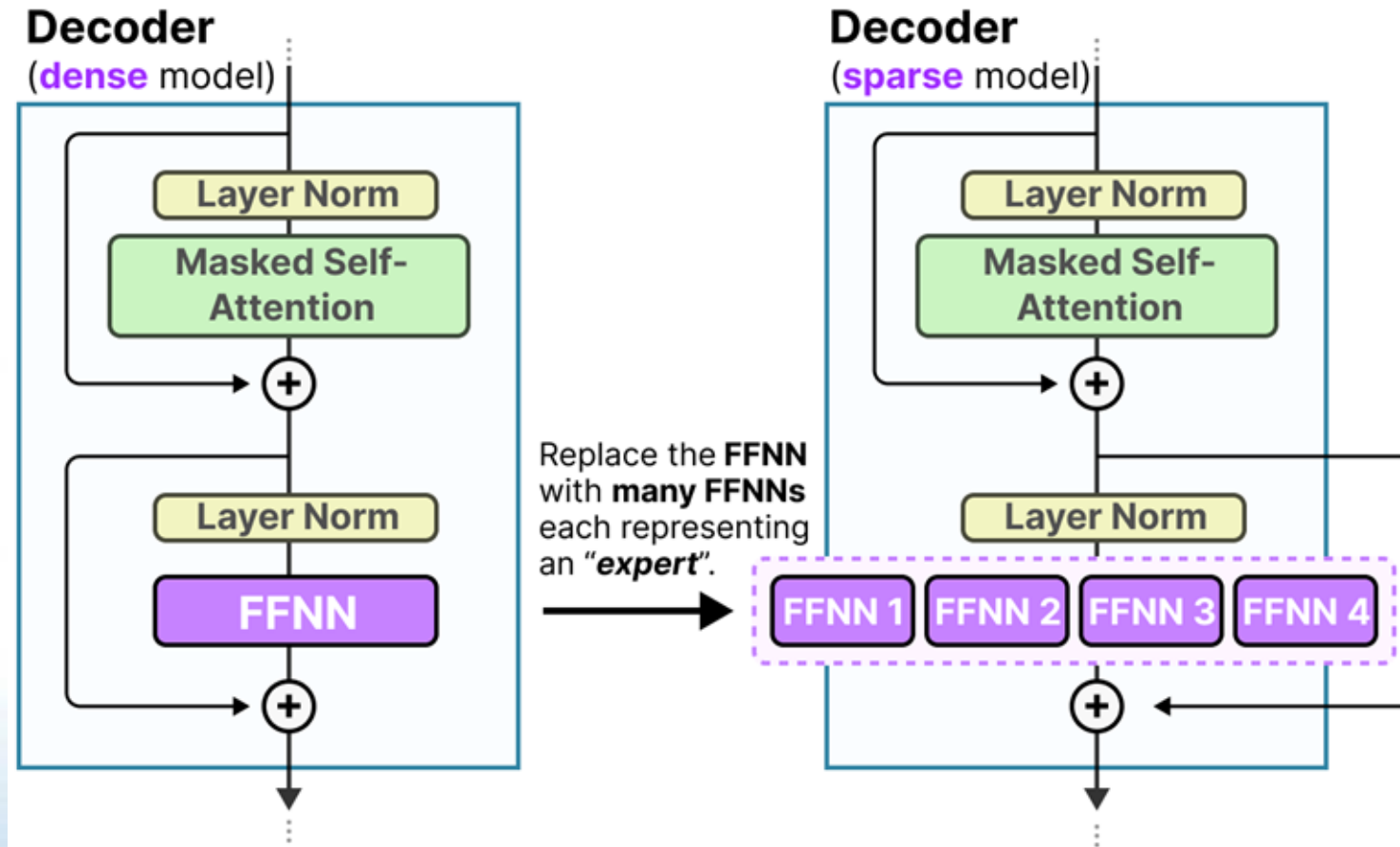


A given text passes through multiple experts before next text is generated

Chosen experts likely differ between inputs which results in different "paths" being taken



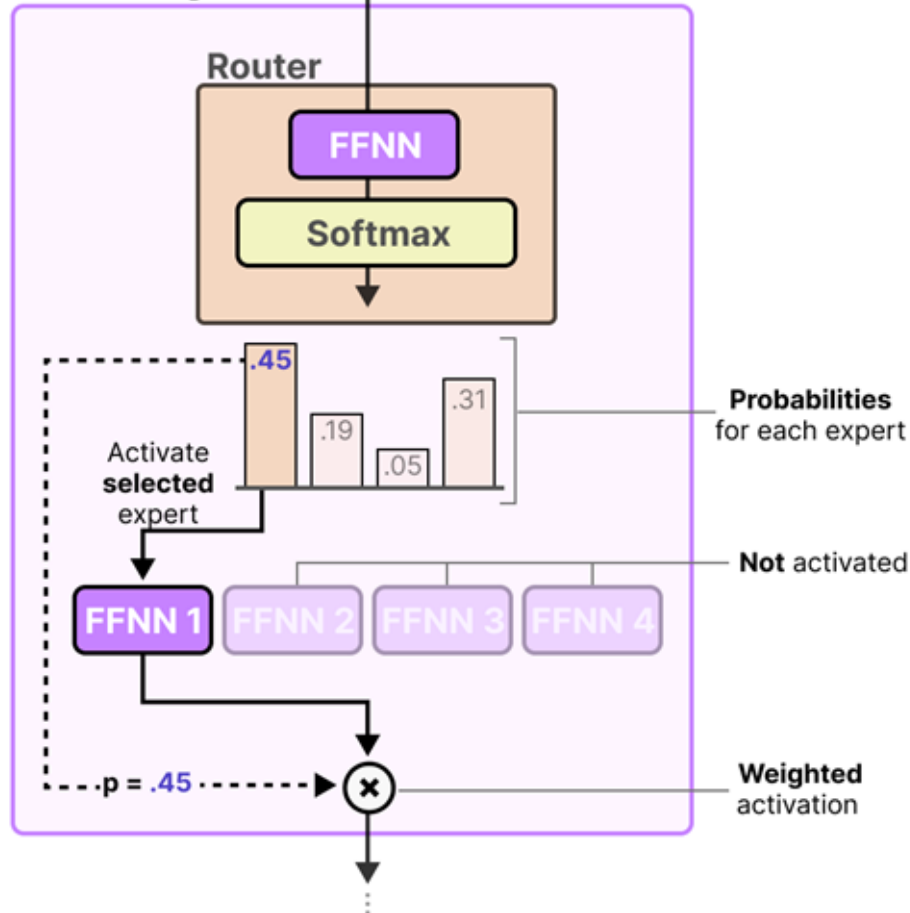
Decoder block with experts





Router to select experts

MoE Layer



output

input

router weights

$$H(\mathbf{x}) = \mathbf{x} * \mathbf{W}$$

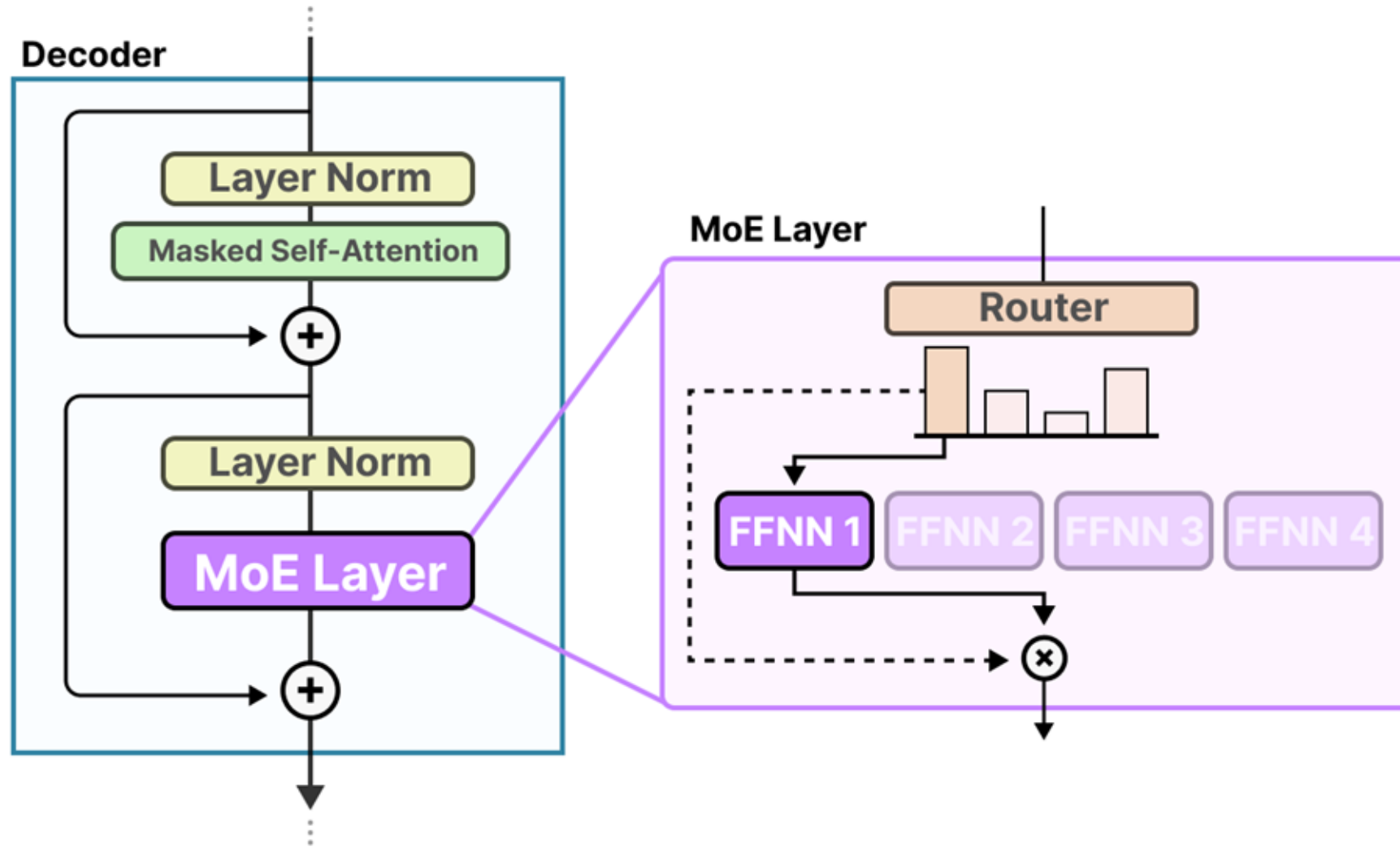
probability distribution per expert

output

$$G(\mathbf{x}) = \text{Softmax}(H(\mathbf{x}))$$

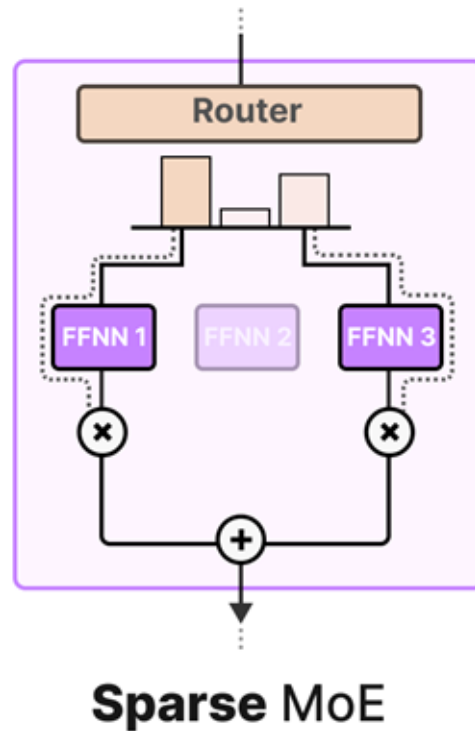
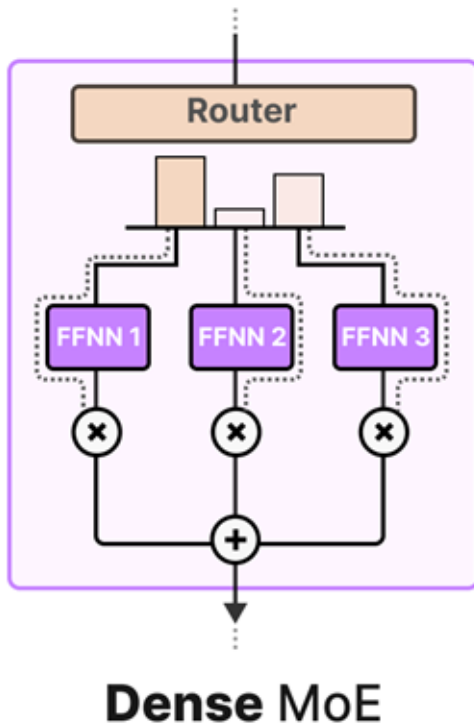


MoE Layer





Sparse vs Dense MoE



Sparse MoE

Sparse MoE output

Router Output

selected Expert

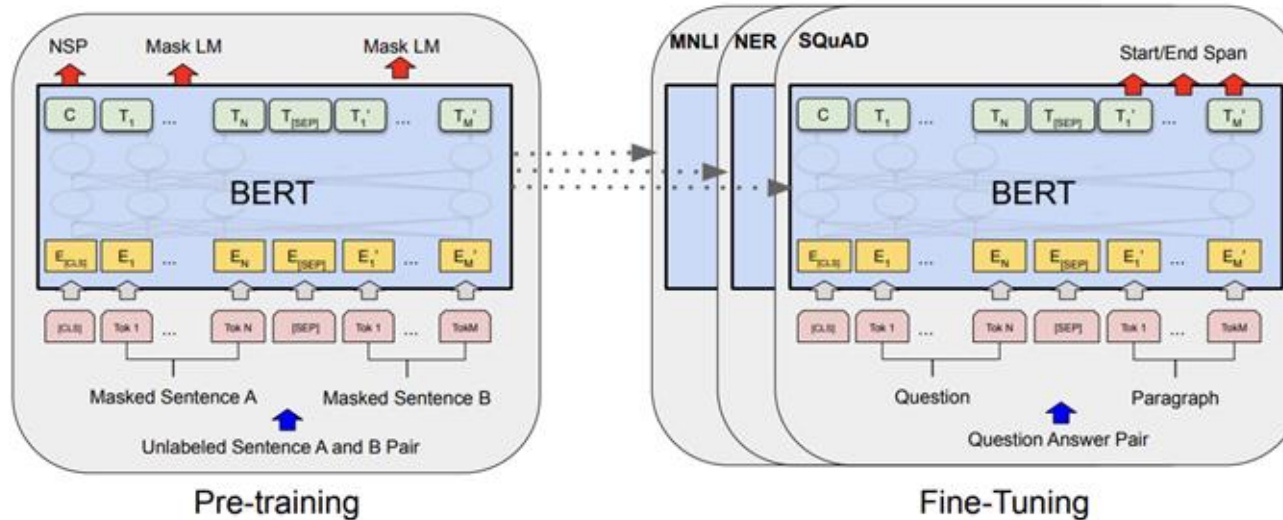
Output per Expert

$$\mathbf{y} = \sum \left(\mathbf{G}(\mathbf{x}) * \mathbf{E}(\mathbf{x}) \right)$$

(only 1 expert is chosen in this example)



Encoder only LLMs

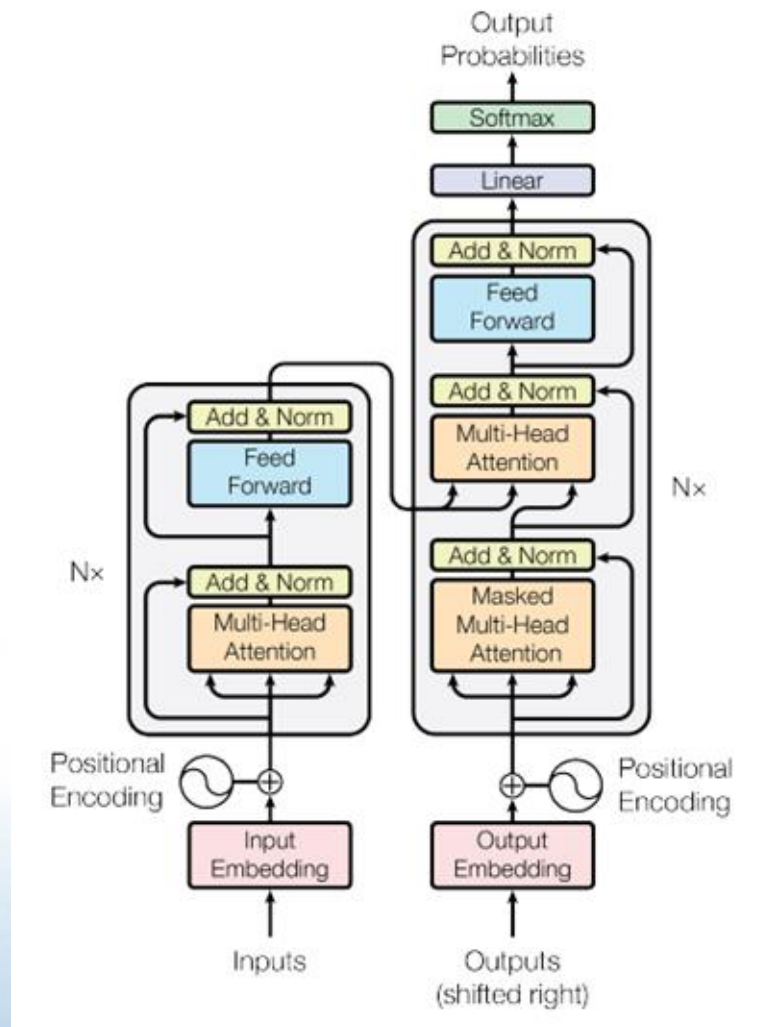


- Good for: classification, sequence tagging (POS tagging, NER), sentiment analysis
- Examples: BERT, RoBERTa, ALBERT, DeBERTa, etc.
- Typically requires fine-tuning for specific tasks
- Cannot generate text (only understand text)

- Sequence of text ☐ fixed length vector representation for text
- Or sequence of vectors for each text



Encoder-Decoder LLMs (seq2seq)



- Original Transformer (Attention is all you need, 2017)
- Other examples: T5, BART
- Good for: machine translation, summarization
- Image captioning (image is fed into encoder)



Decoder only LLMs: Generative models

- GPT, GPT-2, GPT-3, GPT-4, ChatGPT (OpenAI)
- PaLM (Google)
- Chinchilla (Deepmind)
- LLaMA (Meta)



Transfer Learning in NLP

- Used in standard NLP tasks
 - Question answering
 - Text generation
 - Text summarization
 - Named Entity Recognition
 - Key value pair identification
 - ...
- Transfer learning in NLP through
 - Finetuning language models for down stream tasks
 - Generating contextualized word embeddings



Pre-Training Model Architectures in NLP

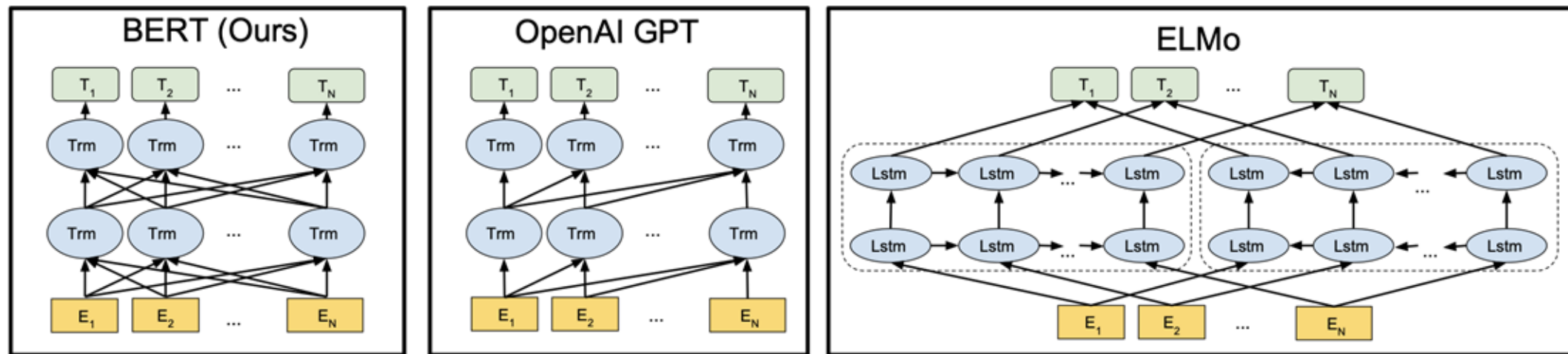


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.



Using Transformer for Transfer Learning in NLP

- Transformers handle long-term dependencies better than LSTMs
- Encoder-Decoder structure of the transformer made it perfect for machine translation
- How to pre-train a language model using transformer-based architecture (with attention mechanisms and faster training due to parallelization) that can be finetuned for downstream supervised learning tasks?



BERT (Bidirectional Encoder Representations from Transformers)

- OpenAI Transformer only trains a forward language model while ELMo's model was bidirectional
- BERT is a transformer-based model whose language model is conditioned on both left and right contexts
- BERT is basically a trained Transformer Encoder stack

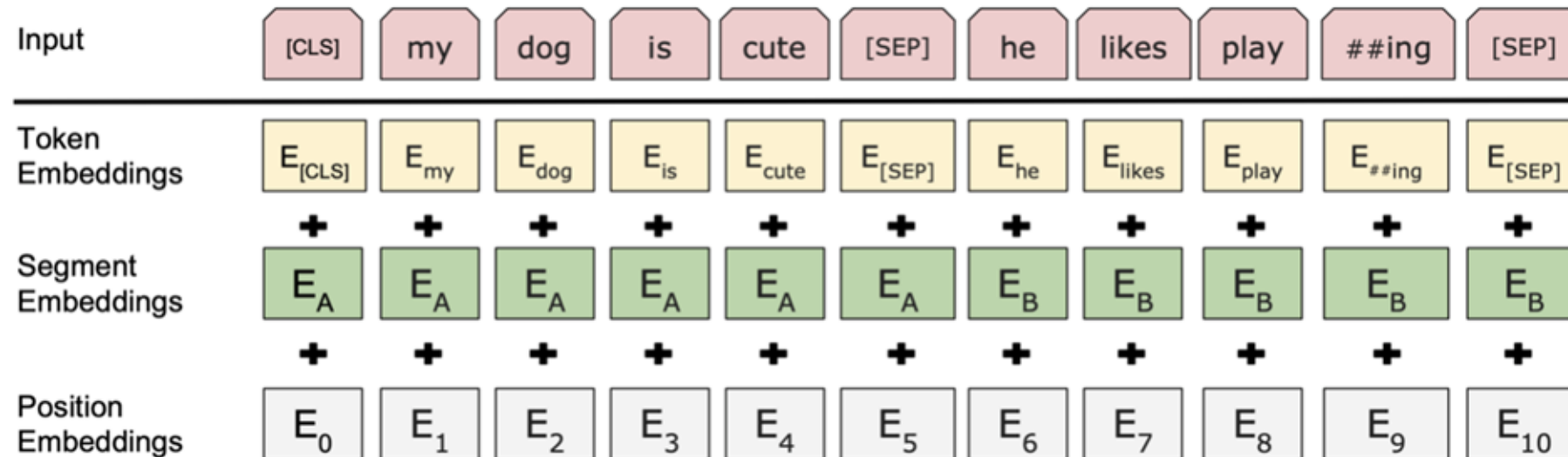


BERT Features

- Training corpus was comprised of two entries
 - [Toronto Book Corpus](#) (800M words), and
 - English Wikipedia (2,500M words)
- Two versions of BERT (L stands for the number of layers, H stands for the hidden size, A stands for the number of self-attention heads)
 - BERT-Base: $L = 12$, $H = 768$, $A = 12$, Total parameters = 110M
 - BERT-Large: $L = 24$, $H = 1024$, $A = 16$, Total parameters = 340M



Input Representation of BERT



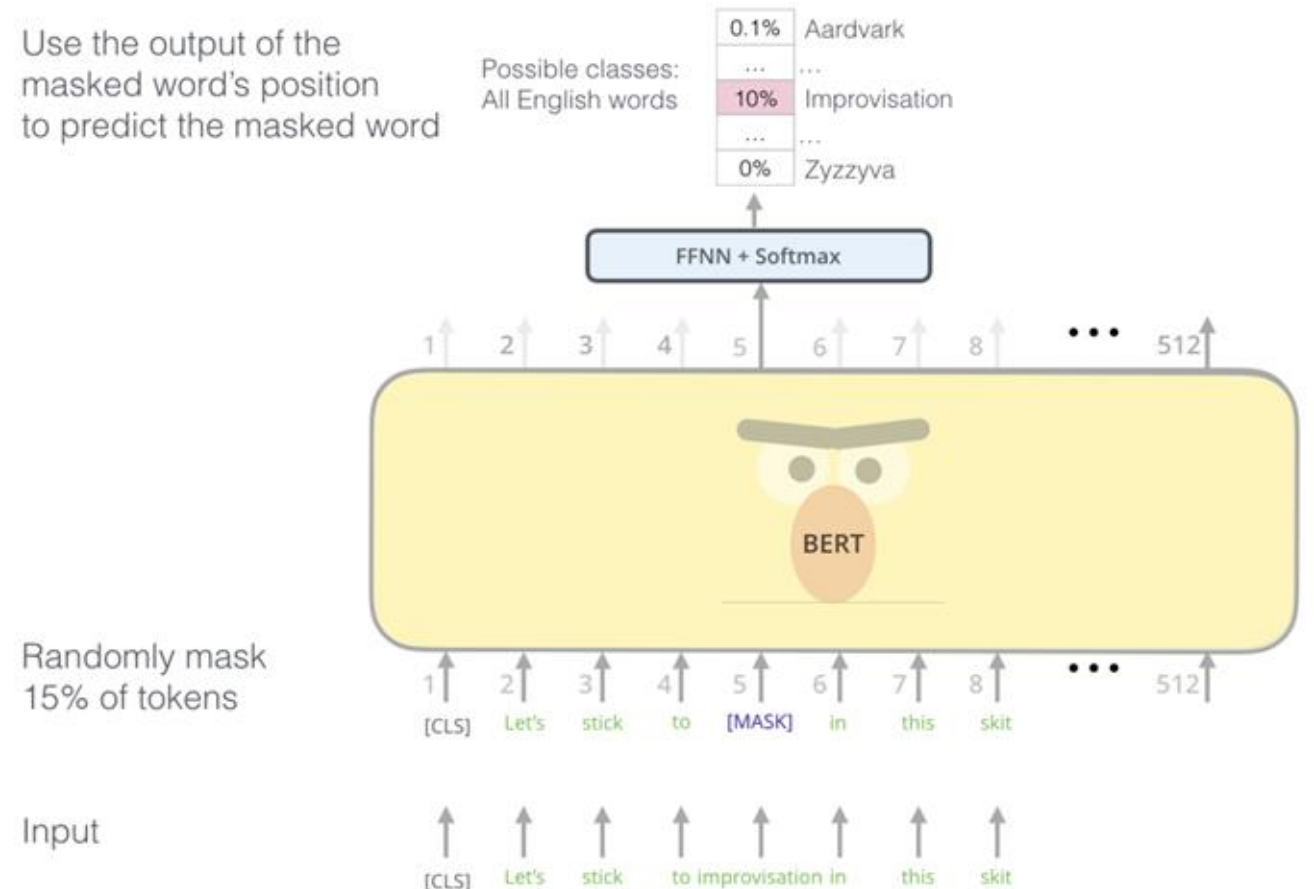


BERT Training – Predicting word at a position

- Masked language model (Masked LM)

That's [mask] she [mask] -> That's what she said

- Randomly masks 15% of tokens in the input and model is trained to predict the masked word



BERT's clever language modeling task masks 15% of words in the input and asks the model to predict the missing word.



BERT Training – Two sentence tasks

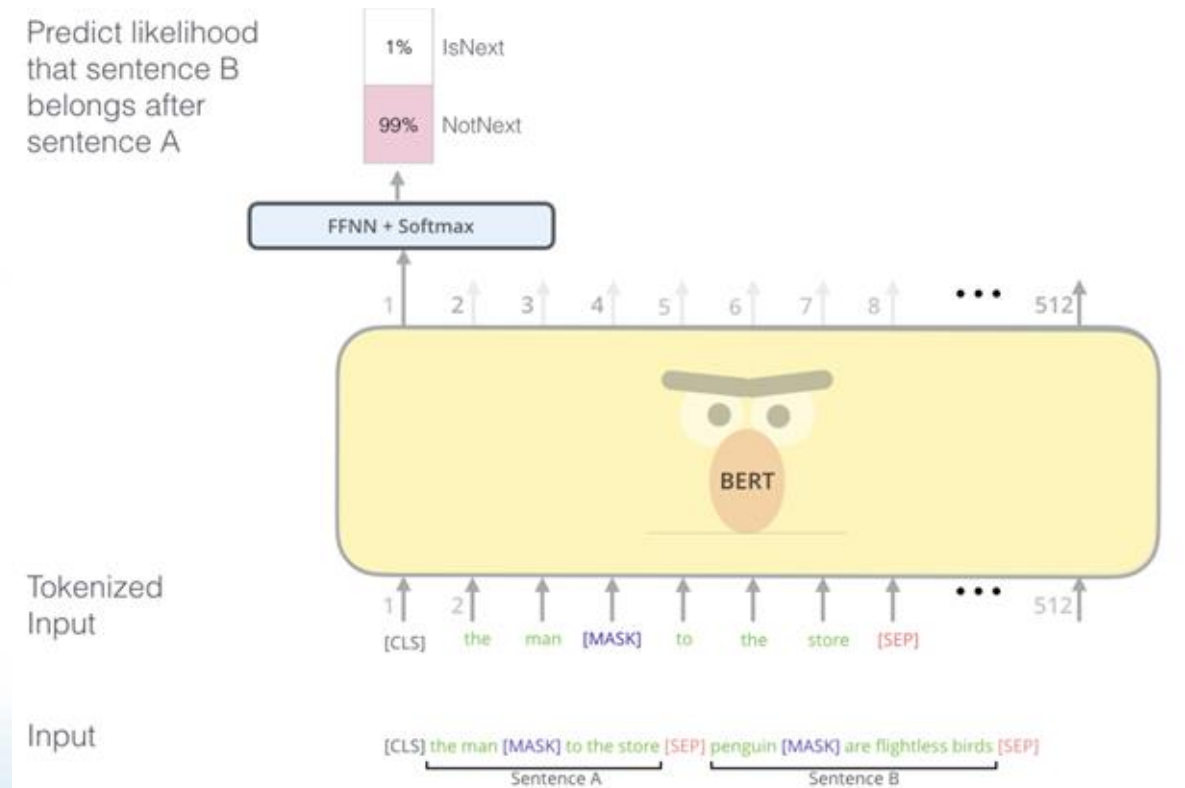
- Next sentence prediction (NSP)
- Task: Given two sentences (A and B), is B likely to be the sentence that follows A, or not? (binary classification)

Input = [CLS] That's [mask] she [mask] . [SEP] Hahaha, nice! [SEP]

Label = IsNext

Input = [CLS] That's [mask] she [mask] . [SEP] Dwight, you ignorant [mask] ! [SEP]

Label = NotNext

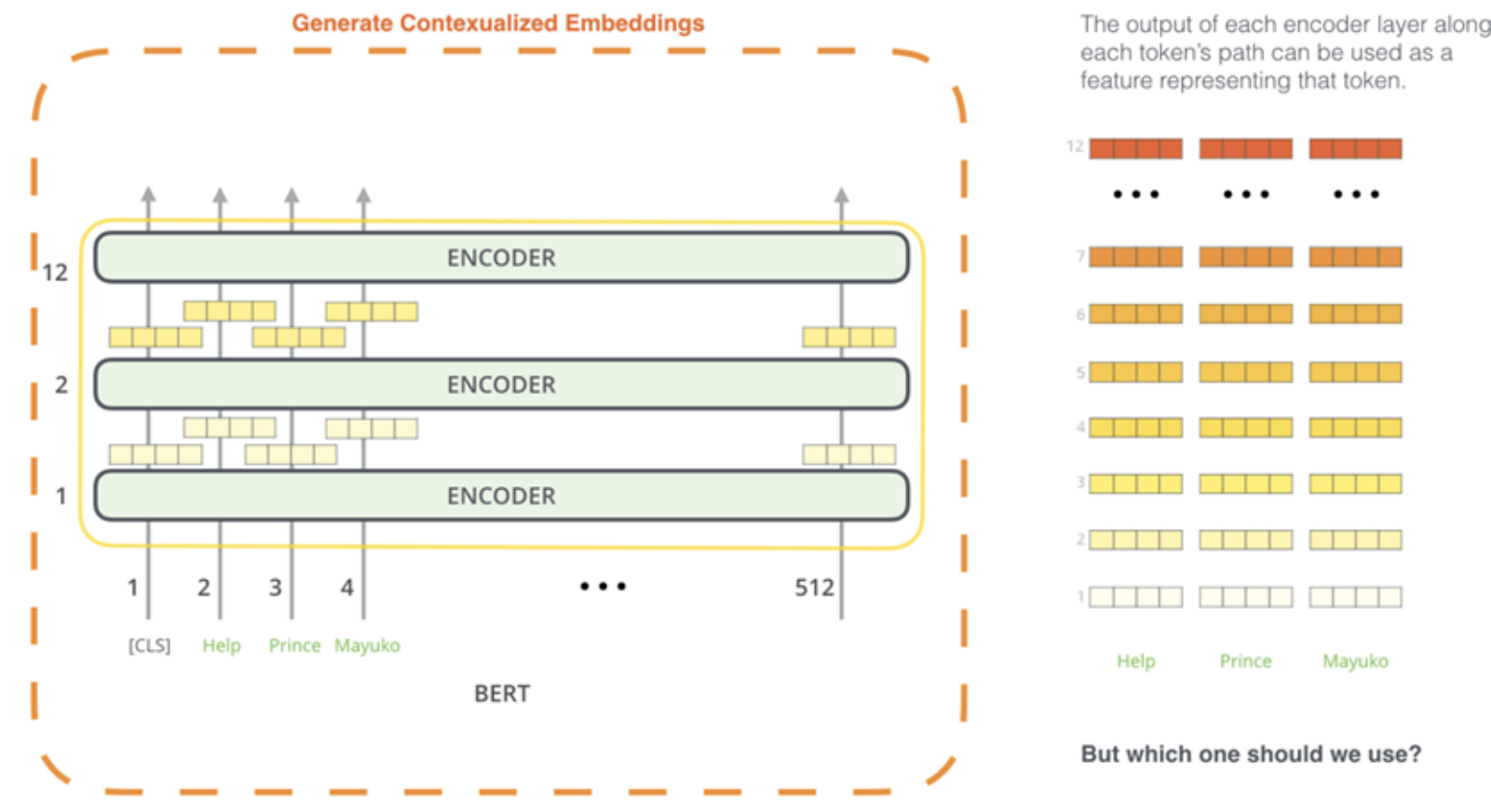


The second task BERT is pre-trained on is a two-sentence classification task. The tokenization is oversimplified in this graphic as BERT actually uses WordPieces as tokens rather than words --- so some words are broken down into smaller chunks.



BERT for Feature Extraction

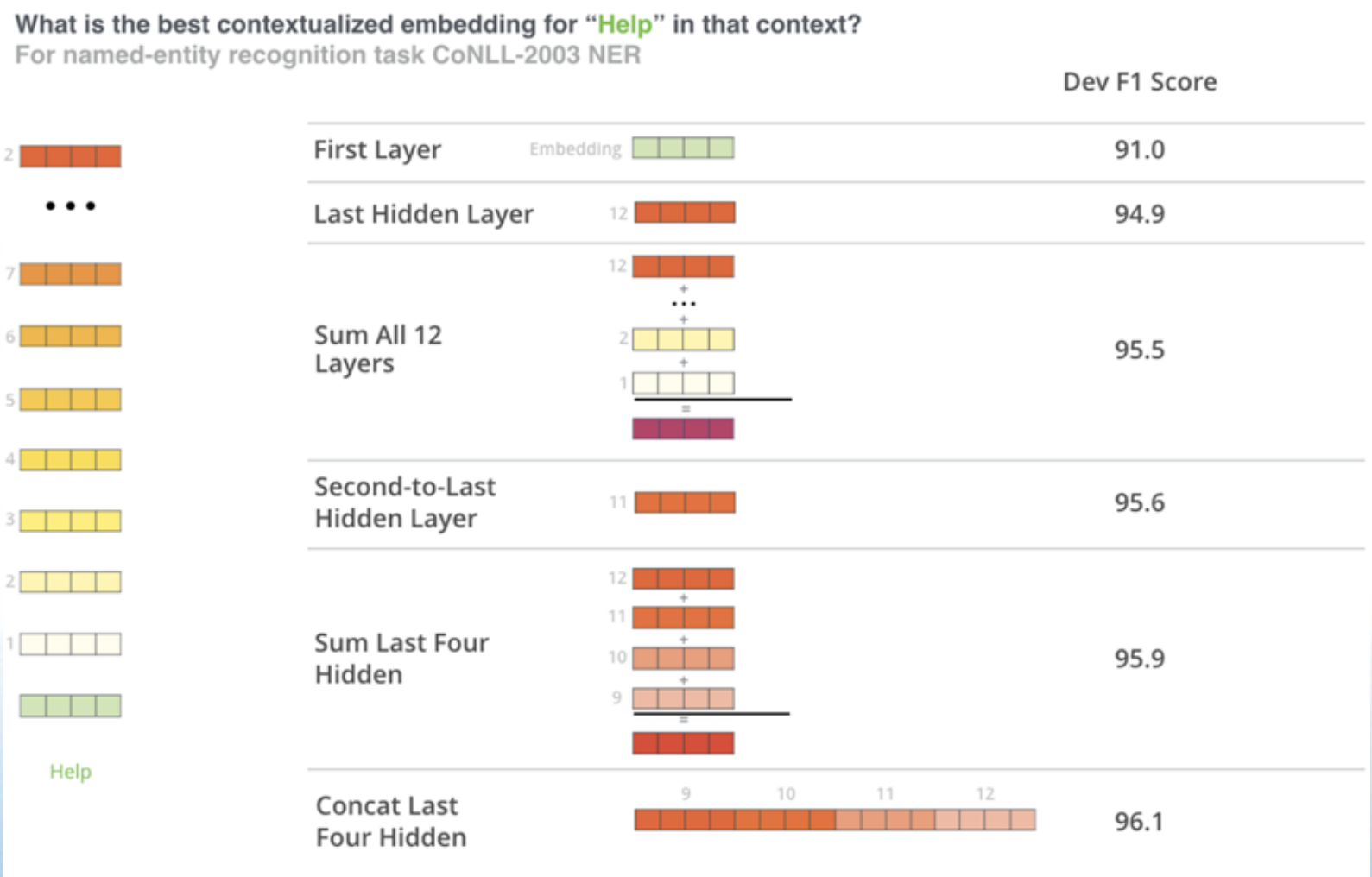
- Using BERT to create contextualized word embeddings



- 12 embedding vectors for each word from the output of 12 encoder layers
- Which embedding(s) to use for a given task?

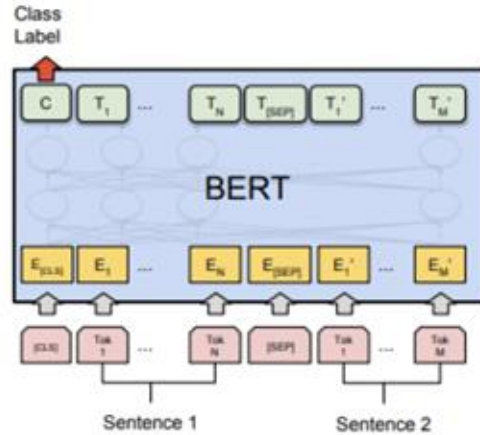


Different ways to create contextualized embeddings for a word using BERT

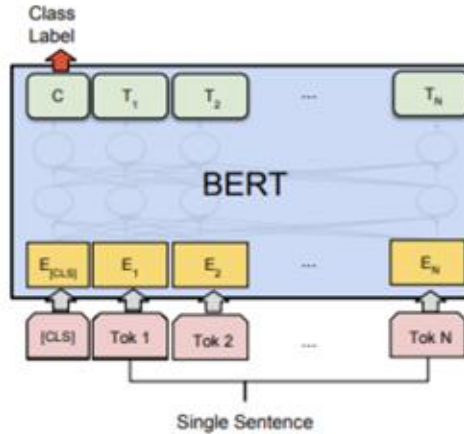




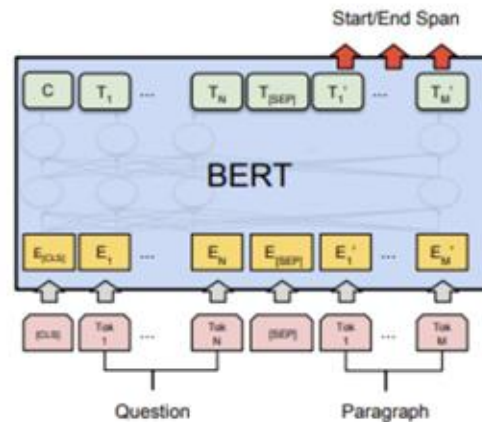
Using BERT for different Downstream Task



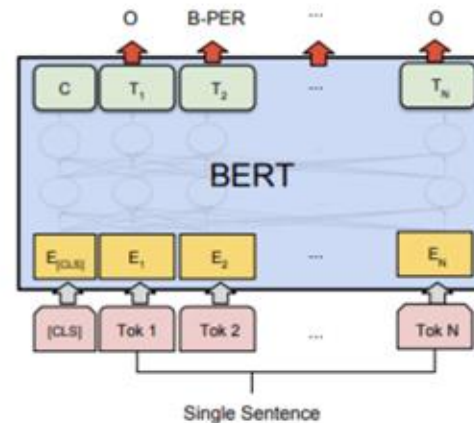
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER



Fine-tuning

Fine-tuning

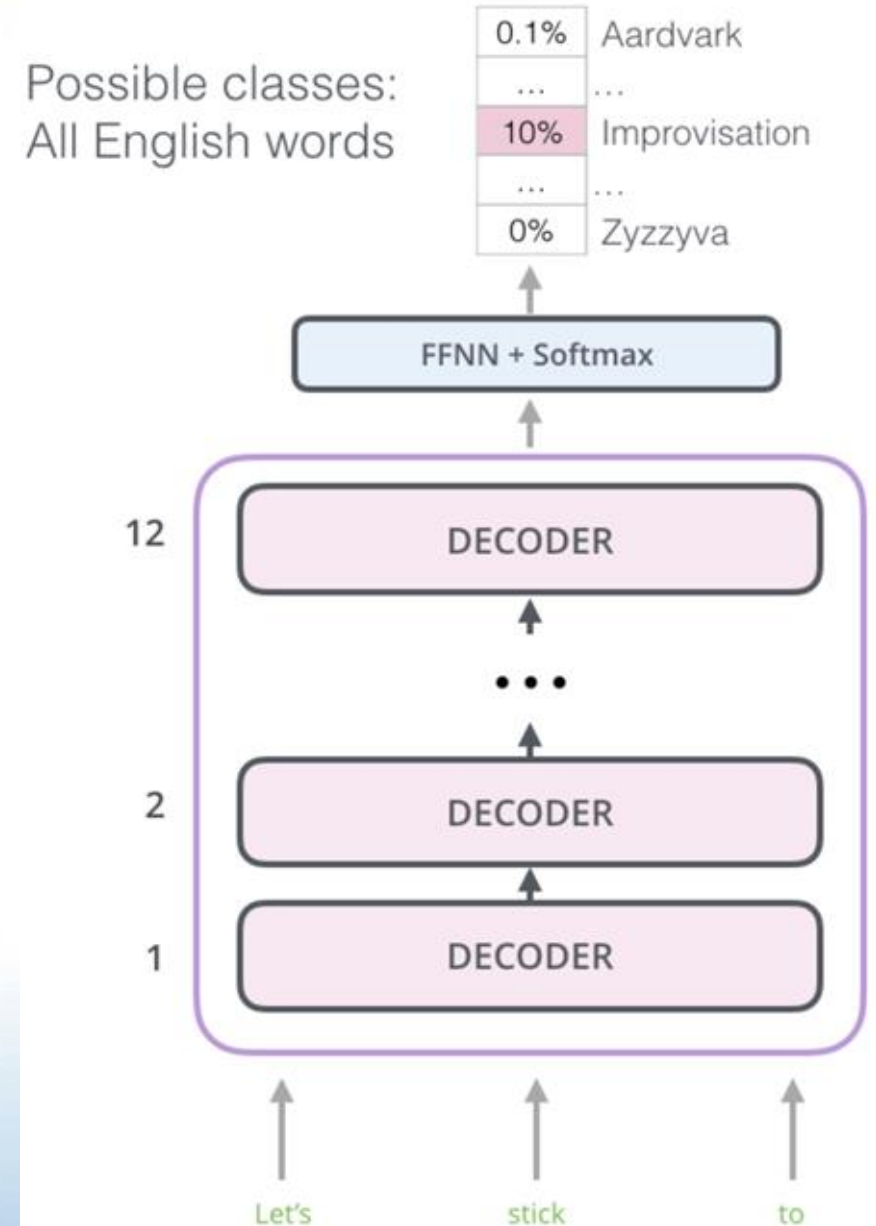
The model is trained via repeated gradient updates using a large corpus of example tasks.





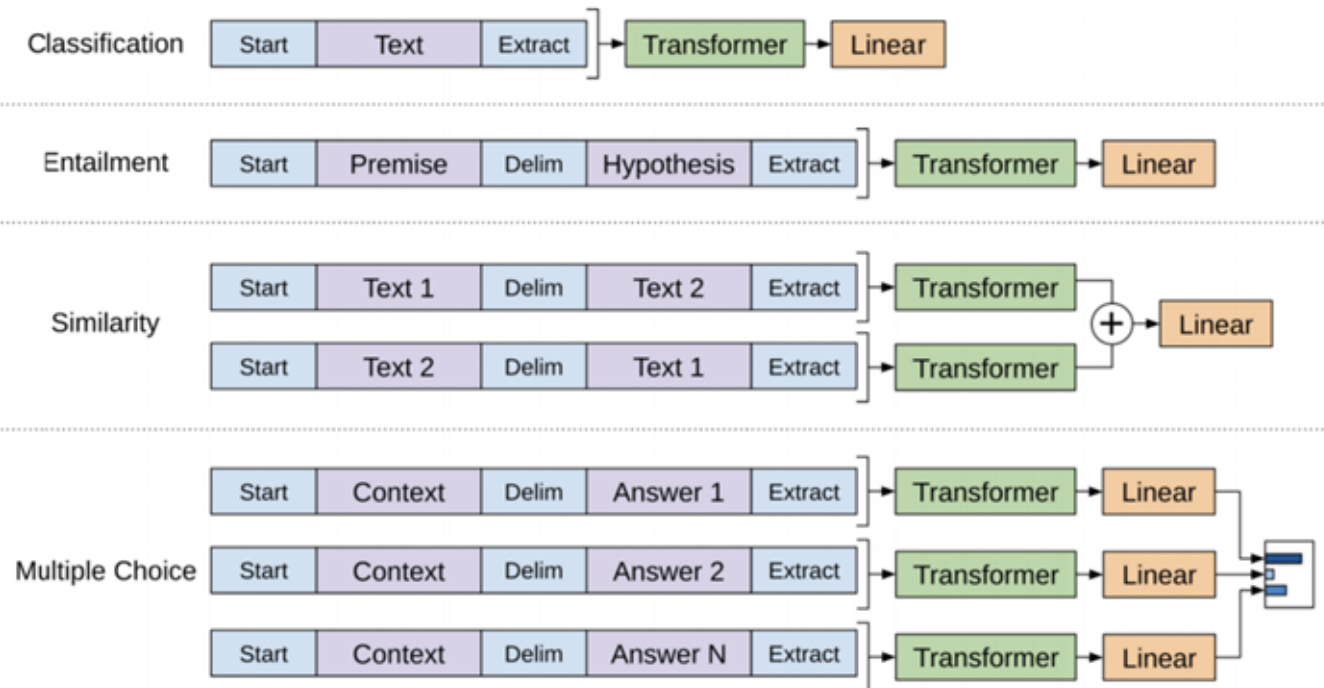
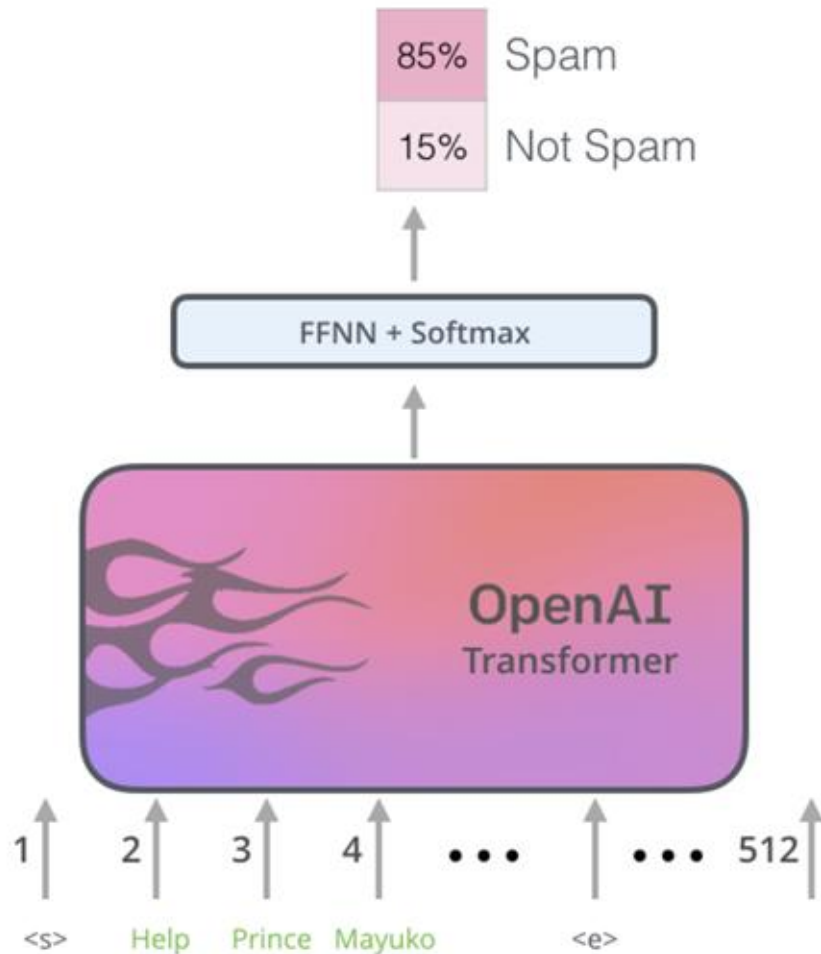
OpenAI Transformer

- Pre-training a transformer decoder for language modeling
- Decoder masks future tokens so can be trained on next word prediction task
- Model has 12 decoder layers stacked
- Only masked self attention layers





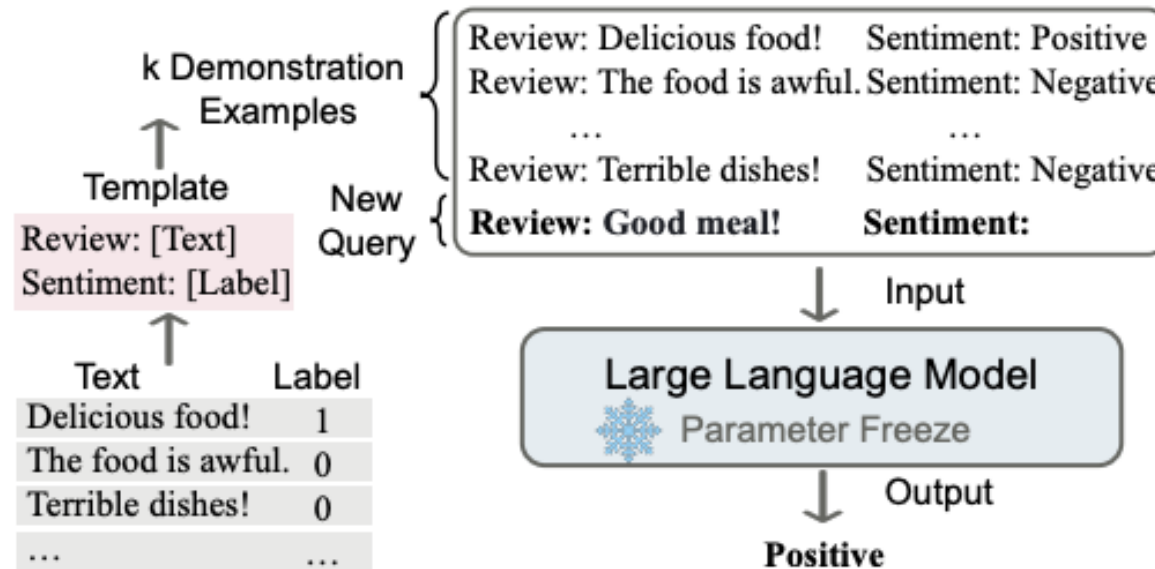
Transfer Learning to Downstream Tasks





In-context learning (ICL)

- Learning from few-examples in the context (input to the LLM)
- Does not require weight updates as in fine-tuning
- Learning from analogy



The three settings we explore for in-context learning

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 cheese => prompt
```

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 sea otter => loutre de mer example
3 cheese => prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1 Translate English to French: task description
2 sea otter => loutre de mer examples
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => prompt
```




Few Shot (FS) Learning

- The model is given description of the task and a few demonstrations of the task at inference time as conditioning but no weight updates are allowed
- Gives K examples of context and completion, and then one final example of context, with the model expected to provide the completion

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← examples
3	peppermint => menthe poivrée	←
4	plush girafe => girafe peluche	←
5	cheese =>	← prompt



One Shot (1S) Learning

- Only one demonstration is allowed, in addition to a natural language description of the task.
- Most closely matches the way in which some tasks are communicated to humans.
- With one example sometimes it is easier to communicate the content or format of a task vs if no examples are given.

One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	sea otter => loutre de mer	← example
3	cheese =>	← prompt



Zero Shot (ZS) Learning

- No demonstrations are allowed, and the model is only given a natural language instruction describing the task
- Some settings zero-shot is closest to how humans perform tasks
- Can be “unfairly hard” in some cases

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

1	Translate English to French:	← task description
2	cheese =>	← prompt

- From OpenAI
- Decode only architecture

GPT-3 family of models

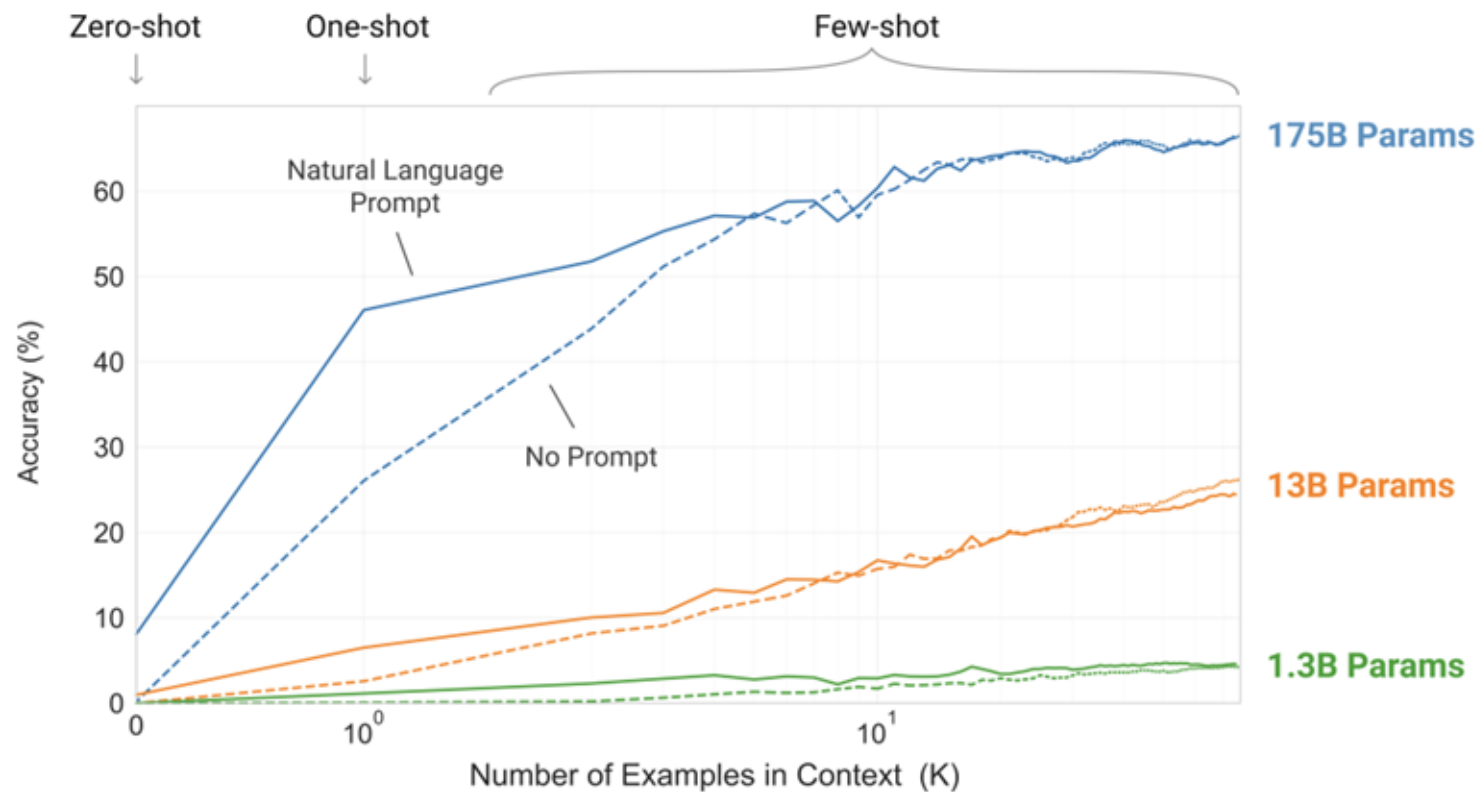
Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

GPT-3 training dataset

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4



GPT-3 performance with ICL



- Task: remove random symbols from a word, both with and without a natural language task description
- Steeper “in-context learning curves” for large models demonstrate improved ability to learn a task from contextual information

Larger models make increasingly efficient use of in-context information.



LLaMa

- From MetaAI
- Decode only architecture

LLaMa family of models

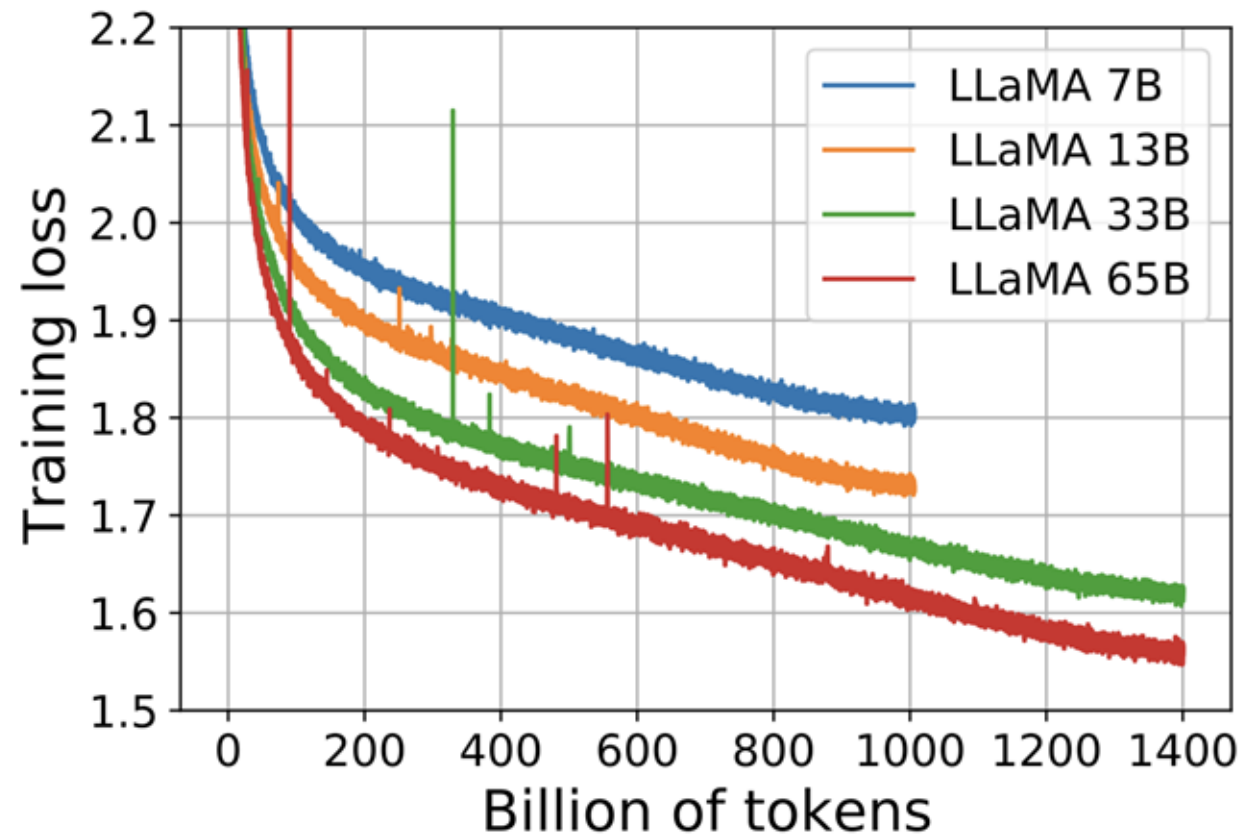
params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

LLaMa training dataset

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB



LLaMa Training



LLaMA-33B and LLaMA- 65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.